МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Московский авиационный институт (национальный исследовательский университет)

Ю.Б. Чечиков, В.Н. Лукин, В.Е. Секретарёв

ОСНОВЫ ЭРГОНОМИКИ ИНТЕРАКТИВНЫХ СИСТЕМ

Учебное пособие

Утверждено
на заседании редсовета
11 февраля 2022 г.

Москва
Издательство МАИ
2023

Чечиков Ю.Б., Лукин В.Н., Секретарёв В.Е. Основы эргономики интерактивных систем: Учебное пособие. — М.: Изд-во МАИ, 2023. — 76 с.: ил.

В пособии обобщен отечественный и зарубежный опыт в области разработки пользовательского интерфейса для прикладных программ. Рассматриваются основные понятия и психологические факторы взаимодействия человека и ЭВМ; описываются психологические основы разработки интерфейса, модели пользовательского интерфейса, метафорический и идиоматический подходы; приводятся критерии хорошего диалога.

Использование приведенного материала позволит понять важность своевременной постановки и решения эргономических проблем для успешного внедрения диалоговых программ.

Для студентов, обучающихся по профилям «Эргономика и интерактивные системы», «Программное обеспечение средств вычислительной техники и автоматизированных систем», «Программирование», «Компьютерные технологии» (направление подготовки 09.03.01 «Информатика и вычислительная техника»).

Рецензенты:

кафедра «Информатика» МТУСИ (зав. каф. канд. техн. наук, доц. *А.И. Волков*);

докт. физ.-мат. наук, профессор *Л.С. Куравский*

ISBN 978-5-4316-1000-4

© Московский авиационный институт (национальный исследовательский университет), 2023

ВВЕДЕНИЕ

Трудовая деятельность человека в автоматизированных системах становится более сложной и напряженной, и возникают трудности в управлении ими. Актуальной в связи с этим становится задача согласования параметров техники с психологическими и физиологическими характеристиками человека. Одним из направлений, изучающих эту важную проблему, является инженерная психология.

Инженерная психология находится на стыке технических и психологических наук, и для нее характерна двойственная область исследования.

Как техническая наука инженерная психология изучает орудия труда и технологические процессы; выясняет, какие требования предъявляются конструкцией инструментов, машин, приборов и особенностями производственных операций к психическим свойствам человека; определяет профессиональную пригодность человека.

Как психологическая наука она изучает психофизиологические процессы и свойства человека с целью определения требований к техническим средствам.

Методом исследования инженерной психологии является *системный подход* — направление методологии научного познания и социальной практики, в основе которого лежит рассмотрение объектов как систем. Системный подход ориентирует исследователя на раскрытие целостности объекта, на выявление многообразных типов связей в нем и сведение их в единую теоретическую картину.

По мере перехода к комплексной автоматизации производства возрастает роль человека как субъекта труда и управления. Человек несет ответственность за эффективную работу всей технической системы, и допущенная им ошибка может привести в некоторых случаях к очень тяжелым последствиям.

Результатом изучения и проектирования таких систем стало создание необходимых предпосылок для объединения технических дисциплин и наук о человеке и его трудовой деятельности, обусловивших появление новых исследовательских задач.

Во-первых, это задачи, связанные с описанием характеристик человека как компонента автоматизированной системы. Речь идет о процессах восприятия информации, памяти, принятия решений, проблемах мотивации, готовности к деятельности, стресса, коллективной деятельности операторов. С точки зрения обеспечения эффективности деятельности человека, важное значение имеют такие факторы, как утомление, монотонность операций, перцептивная (непосредственное отражение действительности органами чувств) и интеллектуальная нагрузка, условия работы, физические факторы окружающей среды, биомеханические и физиологические факторы.

Во-вторых, это задачи проектирования новых средств деятельности, относящихся преимущественно к обеспечению взаимодействия человека и машины. К таким средствам относят визуальные и слуховые индикаторы, органы управления, новые инструменты и приборы.

В-третьих, это задачи системного характера, связанные с распределением функций между оператором и машиной, с организацией рабочего процесса, а также задачи подготовки, тренировки и отбора операторов.

Разработка пользовательских интерфейсов — это отдельное направление, требующее знаний, навыков и образования, отличающихся от знаний, навыков и образования программиста или графического дизайнера.

Согласно советскому энциклопедическому словарю *эргономика* (греч. *ergon* — работа и *nomos* — закон) — это научная дисциплина, изучающая человека и его деятельность в условиях современного производства с целью оптимизации орудий, условий и процесса труда.

Предметом эргономики является трудовая деятельность, основным объектом исследования — системы человек-машина, в том числе и так называемые эргатические системы, т.е. сложные системы управления, составной элемент которых человек-оператор (например, система управления самолетом, диспетчерская служба вокзала, аэропорта).

В области вычислительной техники острая необходимость в достижениях инженерной психологии возникла с появлением интерактивных систем. Под интерактивными системами будем понимать информационно-вычислительные системы, в которых передача и обмен информацией происходят в режиме диалога.

Диалоговый режим — способ взаимодействия пользователя или оператора с ЭВМ, при котором происходит непосредственный и двухсторонний обмен информацией, командами или инструкциями между человеком и ЭВМ. Диалог — это взаимодействие между пользователем и ЭВМ.

Система «человек-машина» (*СЧМ*) — это сложная система, в которой человек-оператор взаимодействует с техническим устройством в процессе производства материальных ценностей, управления, обработки информации. СЧМ не является эквивалентом интерактивной системы, так как интерактивная система — это СЧМ, работающая в режиме диалога. Эргономика рассматривает СЧМ как сложное функционирующее целое, в котором ведущая роль принадлежит человеку. Человека, работающего с помощью машины, принято называть оператором.

Эргономическое обеспечение автоматизированных рабочих мест (АРМ) — создание благоприятных условий для использования АРМ при высокой производительности труда и низкой утомляемости пользователей.

Эргономика решает также ряд проблем, поставленных в системотехнике: оценка надежности, точности и стабильности работы оператора; исследование влияния психологической напряженности, утомления, эмоциональных факторов и особенностей нервно-психической организации оператора на эффективность его деятельности в системе «человек-машина»; изучение приспособительных и творческих возможностей человека.

Эргономика не может эффективно решать стоящие перед ней задачи вне тесной связи с промышленной социологией, социальной психологией и другими общественными науками. Без этих связей эргономика не может ни полноценно развиваться, ни правильно прогнозировать социальный эффект от внедрения разрабатываемых ею рекомендаций. Данная группа наук в определенном отношении опосредствует взаимосвязь эргономики с экономикой.

1. ВЗАИМОДЕЙСТВИЕ ЧЕЛОВЕКА И ЭВМ

1.1. ОСНОВНЫЕ ПСИХОЛОГИЧЕСКИЕ ФАКТОРЫ ВЗАИМОДЕЙСТВИЯ ЧЕЛОВЕКА И ЭВМ

Инженерно-психологические исследования показывают, что новые условия труда приводят к перестройке умственной деятельности человека. В частности, использование ЭВМ существенным образом изменяет структуру процессов мышления. Недооценка роли психологических факторов при построении ЧМС приводит к тому, что человек либо не исчерпывает все предоставляемые машиной средства, либо вообще отказывается их применять,

Так, например, в середине 80-х годов была предпринята попытка автоматизировать одну из ведомственных поликлиник Мосгорисполкома. Работа была поручена одному из предприятий оборонного комплекса. Техническая сторона проекта была сделана грамотно: продумана архитектура сети, конфигурация рабочих мест врачей и медсестер, продублированы особо важные элементы системы. С целью экономии средств было решено не оснащать каждое рабочее место врача принтером, а поставить один скоростной высококачественный принтер в холле каждого этажа. Именно это решение и привело к тому, что спустя месяц после запуска система встала: медперсонал заявил, что отказывается от ее услуг, т.к. бегать за каждой бумажкой в холл и обратно невозможно и поэтому дешевле работать по старой ручной технологии. Систему, конечно, переделали как положено, поставив принтер на каждое рабочее место, но этот пример показывает, что даже технически грамотные системы, не учитывающие интересов пользователей, нежизнеспособны.

Проблема взаимодействия человека и ЭВМ стала особенно острой с появлением ПЭВМ, что привело к увеличению числа пользователей, не являющихся профессиональными программистами, Наибольшие проблемы возникают у людей, имеющих гуманитарный стиль мыш-

ления, потому что они мыслят образами и ожидают, что будут поняты с полуслова, а компьютер требует конкретного мышления с точным указанием всех деталей.

Из всех предлагаемых систем пользователь выберет ту, с которой лично ему работать наиболее комфортно. Очевидно, что под каждого конкретного пользователя создавать отдельную версию интерфейса разработчик не станет, это нереальная задача. А вот учесть основные эргономические принципы комфортного общения и реализовать их — возможно.

Эргономические характеристики системы делятся на физические и психологические. Физическая эргономика занимается изучением соответствия функций системы физиологическим процессам человека — качество изображения, антибликовая защита экрана, удобство расположения клавиш на клавиатуре и т.д. Психологическая эргономика занимается изучением соответствия функций системы психологическим процессам человека. Например, пользователь может прочесть текст, но не может разобрать его смысл из-за непонятного представления информации или из-за того, что скорость выдачи информации слишком велика и у пользователя голова пошла кругом. Разработчики программного обеспечения могут повлиять на психологический комфорт пользователя в ту или иную сторону.

К числу основных психологических факторов взаимодействия человека и ЭВМ относятся: взаимопонимание между человеком и ЭВМ, психологическая готовность к осуществлению взаимодействия, быстрота реакции ЭВМ, удобство общения, отсутствие страхов у пользователя при общении с компьютером. Рассмотрим их более подробно.

1. Взаимопонимание между собеседниками в общем случае возможно при соблюдении условий:

- знание способа общения, с помощью которого собеседники обмениваются информацией;
- наличие некоторого представления о предмете общения.

В проблеме взаимопонимания человека и ЭВМ одним из основных вопросов является разработка языков общения между ними. Появление большого числа пользователей-непрофессионалов требует приближения к естественному языку или перехода к графическим образам используемых понятий.

2. Психологическая готовность к взаимодействию определяется прежде всего наличием у пользователя готового алгоритма решения задачи. Кроме того, она определяется отношением человека к ЭВМ и оценками ее возможностей, формируемыми в процессе этого взаимодействия. Соответствие между этими оценками может принимать противоположные формы: от полного отказа от взаимодействия с ЭВМ до подлинно совместного решения.

Психологическая готовность к взаимодействию зависит также от уровня подготовки пользователя, от наличия посредников (операторов, программистов, консультантов). Например, помощь, полученная от системы в момент возникновения вопроса, или выяснение вопроса откладываются до прихода сопровождающего программиста.

Психологически важно, чтобы пользователь чувствовал контакт с машиной, сам непосредственно с ней общался. Это реализуется в интерактивных системах, в которых устранены посредники между человеком и ЭВМ, введена прямая связь между ними с жестким контролем ошибок и оповещением пользователя о них.

3. Время ожидания ответа ЭВМ оказывает большое влияние на эффективность и структуру деятельности человека. Важность учета времени ответа была осознана и стала изучаться еще в 60-е годы, когда появились первые интерактивные системы. Медленный ответ — это первое, на что пользователь сразу же обратит внимание и будет настаивать на уменьшении времени ответа, потому что быстрый ответ:

- впечатляет;
- создает благоприятное впечатление у тех, кто с машиной работает;
- повышает производительность труда;
- дает ощутимые преимущества по сравнению с прежним способом работы.

Опыт эксплуатации ЭВМ показывает, что разные действия пользователей требуют разных значений времени ответа.

Так, простые, рефлекторного характера (на уровне подсознания) действия требуют быстрых ответов, лежащих в пределах 0.1–0.2 секунды, например, появление символа на экране после нажатия клавиши на клавиатуре (в метро — реакция турникета).

Для ответа на простые команды приемлемо время 0.5–1 секунды (например, выбор пункта из меню и его реализация на следующем экране).

Связанный диалог разрешает задержки 1–2 секунды. Например, когда пользователь отвечает на серию взаимосвязанных вопросов, задержка между ответом пользователя и новым вопросом системы не должна превышать 2-х секунд. Это время является оптимальным для взаимодействия человека с системой, так как при обычном разговоре время реакции собеседника составляет не более 2-х секунд, на которое подсознательно рассчитывает пользователь, работая с компьютером.

Для более сложных действий, в ходе которых происходит анализ полученной информации или формирование запроса, требуется время, равное единицам секунд — от 2-х до 4-х секунд. Перерыв более 4-х секунд нарушает естественный ход мыслей человека и вызывает у него отрицательные эмоции.

Задачи, включающие в себя большой объем обработки, например формирование отчета за период или отработка сложного запроса к базе данных, могут вызвать задержку ответа в десятки секунд. В этом случае надо иметь в виду, что при ожидании интерес к задаче у пользователя сохраняется в течение 20-ти секунд, после чего его внимание переключается на другие задачи и он «забывает» о своем запросе. Искусство разработчика интерактивной системы состоит в том, чтобы удерживать внимание пользователя в течение этих 20-ти секунд.

Если время ответа более 20-ти секунд — это не интерактивная система.

Большое значение имеет также вариация времени ответа. Одной из задач проектирования интерактивной системы является минимизация дисперсии времени ответа для запросов одного уровня сложности. Экспериментально подтверждено, что увеличение разнообразия времени ответа снижает производительность и степень психологического комфорта. Пользователи предпочтут систему, которая постоянно дает ответ через 4 секунды, системе, время ответа которой колеблется в диапазоне от 1 до 6 секунд, хотя среднее время составляет 3.5 секунды.

Увеличение времени ответа сверх ожидаемого может вызвать у пользователя состояния, близкие к стрессовым. (Можно себе представить, что будет твориться в метро, если время реакции турникета в час «пик» начнет колебаться в диапазоне от 0.5 до 4-х секунд.) У пользователя

возникают вопросы, не имеющие отношения к решаемой задаче: исправна ли машина, не «зависла» ли программа и т.д. Реакция его в этих случаях не предсказуема — от хаотического нажатия на все доступные кнопки и клавиши до выключения компьютера из розетки и звонков руководству со словами: «Вашей системой пользоваться нельзя». Во избежание такой реакции лучше предусмотреть сообщения типа «Запрос принят», «Немножко подождите», чтобы пользователь не чувствовал себя брошенным один на один с машиной.

Невозможно сделать систему, у которой все операции укладывались бы во время, не превышающее время реакции пользователя. Но можно сделать такой интерфейс, в котором в течение этого времени будет выдаваться сообщение о том, что информация от пользователя принята и какие действия сейчас выполняет система.

Положительные результаты дает предварительное информирование пользователя о возможных временных задержках ответов ЭВМ. Как правило, это можно реализовать для общесистемных операций, например разметка диска.

Если издержки неизбежны, то в интерфейсе должна быть предусмотрена сообщающая о них обратная связь. Например, в Windows-интерфейсах есть специальный индикатор хода выполнения задачи — Speed Bar — полоса, которая заполняется другим цветом по мере выполнения процесса. Использование такого элемента повышает психологическую комфортность системы, так как, во-первых, пользователь видит, что система приняла от него информацию и, во-вторых, пользователь может заранее оценить длительность процесса. С него снято давление неопределенностью, так как содержимое экрана постоянно обновляется, на подсознательном уровне время ожидания ответа дробится на маленькие этапы и у пользователя возникает ощущение движения к цели.

Если неизвестно, сколько времени будет длиться операция — об этом нужно заранее сказать пользователю. Но и в этом случае можно проявить заботу о пользователе, подав звуковой сигнал после выполнения операции. Звуковым сигналом удобно сопровождать сообщения, рассылаемые по терминалам в системах реального времени, чтобы пользователь мог как можно быстрее на них среагировать. Например, в реализованной информационно-измерительной системе для биохи-

мической лаборатории «Биолаб» после ввода информации с бланков-заказов в систему регистратура рассылает на рабочие места лаборантов сообщение о частично введенном диапазоне заказов, чтобы лаборанты могли отобрать соответствующий биоматериал и приступить к исследованиям. Ведь никто из лаборантов специально не сидит у терминала, ожидая сообщения из регистратуры. Как правило, они заняты подготовительной работой и звуковое сообщение бывает как раз кстати.

Как ни удивительно, задержка времени ответа для минимизации дисперсии может действительно повысить производительность труда и увеличить чувство удовлетворенности системой. Обычно это применяется в многопользовательских системах с разделением времени, чтобы сгладить скачки времени реакции в связи с колебаниями нагрузки на систему в течение суток. В этом случае при слабой нагрузке ответ принудительно задерживают до среднего времени ответа, а при пиковой нагрузке ослабляют запрос, уменьшая, например, диапазон поиска по датам или номерам. Также используют задержку времени ответа при смене системы на более мощную. Пока нагрузка не стабилизировалась, запускают циклическую программу с высоким приоритетом, которая замедляет реакцию системы. По мере роста нагрузки приоритет программы уменьшают.

Разработчик должен знать, что если у пользователя общение с системой не вызывает внутреннего сопротивления, — значит система спроектирована правильно.

Преодоление безотчетного страха перед компьютером (киберофобии) — весьма тонкая и деликатная проблема. Панический страх, что после каких-либо действий или нажатия на клавиши компьютер сломается или уничтожит все результаты работы, напрочь блокирует все контакты пользователя с машиной. Частичное решение этой проблемы требует такта и терпения. В первую очередь, надо объяснить пользователю принципы работы системы и показать, что она устойчива, живуча и защищена от нелепых ошибок. Надо попросить пользователя озвучить свои опасения, а потом рассказать ему что́ будет в той или иной ситуации на самом деле. Подобная работа требует большого такта и уважения к чужим недостаткам. Ни в коем случае нельзя подтрунивать над человеком или обсуждать его проблемы с кем-то из сослуживцев — необходимо хранить клиентскую тайну.

Очень хорошие результаты дает использование компьютерных игр на начальном этапе обучения — пользователь перестает беспокоиться за результаты своей работы, осваивает клавиатуру, начинает понимать компьютер. Собственно, завоевание жизненного пространства персональными компьютерами началось именно с игровой начинки первых PC. В частности, фирма Apple сумела закрепиться на рынке благодаря удачно подобранным играм.

1.2. ПОНЯТИЕ ИНТЕРФЕЙСА ПОЛЬЗОВАТЕЛЯ

Основным звеном интерактивных систем является интерфейс человек-машина. Человеко-машинный интерфейс (ЧМИ) — это комплекс аппаратных, программных, методических и организационных средств, обеспечивающих взаимодействие человека и ЭВМ. Поскольку основным звеном ЧМИ является человек, очевидно, что при разработке интерфейса человек-машина особое внимание должно быть уделено изучению таких психологических свойств человека, как восприятие, память, внимание и мышление. Именно от этих свойств зависит качество информационного взаимодействия человека с машиной.

Например, человек может допускать ошибки не из-за профессиональной непригодности или некомпетентности, а из-за ограниченности психофизиологических возможностей. В частности, скорость передаваемых ему данных превышает возможности органов чувств, или форма представления информации трудна для осмысления, или выводимое на экран изображение недостаточно контрастно для данного уровня освещенности.

Обмен информацией между пользователем и ЭВМ осуществляется передачей сообщения. В ответ пользователь получает подсказки или справки; информационные сообщения, требующие ответа; приказы, требующие действия; сообщения об ошибках и другую информацию.

Интерфейс пользователя — элементы и компоненты программы, которые способны оказывать влияние на взаимодействие пользователя с программным обеспечением. В том числе:

– средства отображения информации, отображаемая информация, форматы и коды;

– командные режимы, язык пользователь-программа;
– устройства и технологии ввода данных;
– диалоги, взаимодействие и транзакции между пользователем и компьютером;
– обратная связь с пользователем;
– поддержка принятия решений в конкретной предметной области;
– порядок использования программы и документация на нее.

Пользовательский интерфейс часто понимают только как внешний вид программы. Однако на деле пользователь воспринимает через него всю программу в целом, а значит, такое понимание является слишком узким.

В действительности пользовательский интерфейс объединяет в себе все элементы и компоненты программы, которые способны влиять на взаимодействие пользователя с программным обеспечением.

Следует отметить, что проблемы пользовательского интерфейса не решаются логикой — это не математика или программирование, где есть готовые отлаженные решения. При работе с пользователем главную роль играет способность *почувствовать* что́ ощущает пользователь в тот или иной момент во время общения с программой. Эту способность в себе надо постоянно развивать.

Само понятие «человеко-машинное взаимодействие» говорит о наличии человека в качестве одной из сторон взаимодействия. Однако это отнюдь не какой-то абстрактный «обычный человек» или «усредненный пользователь». **Каждая система, каждый интерфейс предназначен для использования одной, а чаще — несколькими конкретными категориями пользователей, которые обладают определенными характеристиками.**

«Удобных» интерфейсов не существует. Точнее, не существует интерфейсов, которые были бы одинаково удобны в использовании для абсолютно всех людей, равно как и не существует обуви, подходящей всем. Понимание этого факта является очень важным при разработке интерфейса. Поэтому для дизайнера интерфейсов совершенно необходимо знать, кто будет пользоваться системой, которую он проектирует, и в каких условиях это будет происходить. Причем знать это нужно уже на начальных этапах разработки интерфейса и программной системы в целом.

Таким образом, проектирование интерфейса не является частью процесса разработки программного обеспечения, а должно быть частью процесса создания спецификаций на систему.

1.3. РАСПРЕДЕЛЕНИЕ ФУНКЦИЙ МЕЖДУ ЧЕЛОВЕКОМ И ЭВМ

Если считать, что программирование — это процесс отображения элементов предметной области на вычислительные средства, то становится ясным сам принцип использования техники, и в частности компьютера, в деятельности человека: то, что мы смогли о себе осознать, формализовать и промышленно воспроизвести — можно поручать технике, компьютеру. Все остальное должен делать человек. Человек должен решать задачи, не поддающиеся формализации, требующие нестандартного мышления и высокой ответственности.

На человека в интерактивной системе желательно возлагать следующие функции:

1. Распознавание ситуации в целом по ее многим взаимосвязанным характеристикам, а также в случае неполной информации о ситуации.

2. Осуществление обобщения на основе отдельных фактов, особенно при неполной информации (индуктивное мышление — от частного к общему).

3. Решение задач, в которых отсутствует алгоритм или нет четко выраженных правил переработки информации (интуитивные способности человека).

4. Решение задач, в которых требуются гибкость и адаптация к изменяющимся условиям, особенно ситуаций, которые трудно заранее предвидеть.

5. Решение задач с высокой степенью ответственности, в которых велика цена ошибки (включение реактора, постановка диагноза).

Машине целесообразно поручать следующие задачи:

1. Выполнение всех видов математических расчетов.

2. Выполнение однообразных, повторяющихся операций по известному алгоритму.

3. Хранение большого объема информации для использования в расчетах и выдачи по запросам.

4. Решение задач, требующих дедуктивного мышления, т.е. получение решений для частного случая на основе общих правил.

5. Выполнение действий, требующих быстрой реакции на команду.

6. Работа в течение продолжительного периода времени с устойчивой связью «стимул-реакция».

Человек, являясь звеном интегральным и универсальным, превосходит машину: в ассоциативном мышлении (образное мышление), в применении интуиции (генетический опыт), в возможностях обобщения; он обладает большой гибкостью программирования и перепрограммирования поведения, способностью к самообучению и изобретательности; различает слабое соотношение сигнал/шум; в стрессовых ситуациях способен превышать свои возможности или, наоборот, терять их.

ЭВМ превосходит человека в быстродействии, точности вычислений, в объеме и скорости обращения к оперативной памяти, в стабильности, надежности, длительности непрерывной работы.

С другой стороны, по своим возможностям человек и ЭВМ дополняют друг друга. При распределении функций между ними возможен выбор одного из двух вариантов:

1. Человек контролирует машинный процесс решения задачи и утверждает решение.

2. Процесс решения задачи осуществляется последовательно с участием человека и ЭВМ.

Выбор варианта обусловливается характером решаемой задачи и обосновывается анализом возможностей человека и машины, для чего строится модель функций системы в виде списка функций и для каждой функции решается кому — человеку или машине — она должна быть передана.

Следует учитывать и такой фактор, как отказы ЭВМ. Если отказ машины может привести к опасным последствиям, то человек должен иметь возможность вручную выполнить необходимые действия, даже если при этом качество управления будет ухудшено.

2. ПОЛЬЗОВАТЕЛИ, УДОБСТВО, РАЗРАБОТКА

2.1. КЛАССИФИКАЦИЯ ПОЛЬЗОВАТЕЛЕЙ

При рассмотрении интерфейса «человек-машина» (ИЧМ) важное значение приобретает изучение пользователей. Необходимость такого изучения достаточно очевидна: коль скоро мы говорим о проектировании ИЧМ, обеспечивающего эффективное взаимодействие с пользователем, то необходимо знать на какого пользователя ориентироваться. Обычно различают пять типов пользователей.

1. *Случайный* (*«наивный»*) *пользователь*. Работа в диалоге не связана с основной деятельностью, диалог непредсказуем. Наивного пользователя можно встретить в банках около терминалов по оплате ЖКХ.

2. *Параметрический пользователь*. Взаимодействие с ЭВМ — составная часть работы, регламентированная четкими правилами, и соответствует вполне определенной области занятости (сотрудники регистратуры в больницах, гостиницах, операторы, диспетчеры).

3. *Аналитики и исследователи* (высшее руководство фирмы, научные работники). Взаимодействие с ЭВМ — составная часть работы, но их потребности непредсказуемы. Это наиболее хлопотный тип пользователя для программиста — отказать в просьбе не всегда возможно, равно как и выполнить ее.

4. *Прикладные программисты*. Взаимодействие с ЭВМ является основной частью их работы. Разрабатывают программы с использованием специальных инструментальных сред разработчика. Все их усилия направлены на решение задач, поставленных пользователями первых трех типов. Получают удовольствие от общения с пользователем.

5. *Системные программисты*. В совершенстве знают аппаратные и программные возможности ЭВМ. Работают совместно с прикладными программистами, но тяготеют к работе с аппаратурой, а не с людьми.

2.2. «НАИВНЫЙ» ПОЛЬЗОВАТЕЛЬ

Рассмотрим «наивного» пользователя несколько подробнее, т.к. многие системы, предназначенные для массового пользования, ориентированы именно на эту категорию. Из опыта работы можно выявить их наиболее характерные свойства.

1. Машина им нужна для выполнения специфических задач, и они будут оценивать ее эффективность в соответствии со способностью решать именно эти задачи.

2. У них нет глубоких знаний по работе с вычислительной техникой вообще или с конкретной системой и, как правило, они не стремятся к таким знаниям.

3. Пытаясь минимизировать необходимость обучения, они стараются также минимизировать время и усилия, затрачиваемые на пользование вычислительной системой.

4. Понимают, что система защищена от нелепых ошибок, и поэтому не очень сильно контролируют свои действия (как правило, у таких пользователей руки работают быстрее головы). Или диаметрально противоположная ситуация: не нажимал ни одной клавиши, чтобы случайно ничего не сломать в системе.

5. Предпочитают не проявлять инициативу, а подчиняются тому, чтобы их вели через систему.

6. Минимальная квалификация в машинописи, трудности при вводе текстовой информации, много опечаток. Поэтому велика потребность в переходе к графическим образам, выбору информации, а не к ее вводу.

7. Надеются, что системой будет легко пользоваться и она решит все задачи, стоящие перед пользователем. Идеал системы — нажал одну кнопку и все получил.

В итоге «наивный» пользователь нуждается в доступе к набору машинных средств, соразмерных его потребностям в обработке информации, хочет иметь средства простые для понимания и использования; нуждается в сопровождении (системе помощи) при работе с диалоговой системой.

2.3. УДОБСТВО ДЛЯ ПОЛЬЗОВАТЕЛЯ

Понятие «удобство для пользователя» часто используется в литературе, но редко бывает четко определено в деталях, и, видимо, не сущест-

вует согласия относительно его смыслового значения ввиду большого субъективизма данного понятия. Некоторые трактовки этого понятия:

1) качество, соответствие требованиям пользователя;

2) способность системы реагировать в соответствии с ожиданиями пользователя;

3) обеспечение легкого доступа к машине;

4) система гарантирует физический и психологический комфорт, обеспечивает повышенную степень удовлетворенности работой;

5) обеспечение минимальных интеллектуальных усилий для пользователя.

Очевидно, что из этих определений невозможно вывести какие-либо конкретные указания для проектирования системы. Чтобы разобраться с «удобством для пользователя», введем понятие *операционная сложность*. Операционная сложность означает сложность работ, которые должен выполнить пользователь, чтобы достичь решения поставленной задачи. Различают объективную операционную сложность, являющуюся действительной сложностью поставленной задачи, и субъективную операционную сложность, которая отражает личные ощущения пользователя, связанные с объективной операционной сложностью, которую ему приходится преодолевать.

Пользовательский интерфейс сильно влияет именно на субъективную операционную сложность. Будем считать, что интерактивная система удобна для пользователя, если пользователь чувствует, что субъективная операционная сложность общения с ней мала. Поэтому при разработке человеко-машинного интерфейса всегда присутствует задача снижения субъективной операционной сложности общения. Когда пользователь впервые садится за работу с системой, чего он больше боится: сложности задачи или сложности общения с системой? Как правило, последнего.

Рассмотрим, от чего зависит субъективная операционная сложность системы. Субъективная операционная сложность отражает личные ощущения пользователя, связанные с преодолеваемой операционной сложностью. Субъективная операционная сложность определяется тремя следующими факторами:

1. Объективной операционной сложностью действий, которые должен выполнить пользователь, чтобы достичь решения поставленной задачи.

2. Характеристиками пользователя — его квалификацией, навыками общения с ЭВМ, свойствами личности и т.д.

3. Качеством интерфейса.

Следует отметить, что удобство пользователя не тождественно эффективности. Система может быть эффективной, но неприятной в общении или удобной, но неэффективной. Например, библиографическая информационная система может давать отличные развернутые справки, но требовать утомительного детального ввода подробного запроса. Или быть простой в подготовке данных, но быть неспособной к выполнению ряда функций.

Что же должен сделать и учесть разработчик, чтобы его система была удобной для пользователя? Во-первых, надо выяснить, каковы ожидания пользователя; во-вторых, знать и учитывать основы психологии мышления пользователя и, в-третьих, грамотно разрабатывать систему, содержащую интерфейс пользователя.

2.4. ОЖИДАНИЯ ПОЛЬЗОВАТЕЛЯ

Как же выяснить ожидания пользователя?

Когда пользователь садится за работу с программой, он уже имеет некоторые представления о том, как будет работать программа. Если ему уже приходилось работать с подобной программой, то он будет думать, что и эта программа работает точно так же, и будет следовать определенным принципам. Пользователь может догадываться, как будет работать интерфейс. Собственное понимание пользователем того, что́ делает программа и как с ней общаться, называют *моделью пользователя*. Модель опирается на представляемые ожидания, и пользователь надеется, что интерфейс будет:

1) интуитивно понятным и обеспечит минимум затрат на обучение;

2) легким в общении (все под рукой в нужное время);

3) максимально соответствующим принятой технологии работы: все, что можно было сохранить из ручной технологии — здесь есть.

У программиста при проектировании системы тоже имеется своя модель — его понимание того, что нужно пользователю. Если эти две модели совпадают, то система имеет высокие шансы не обмануть пользователя и даже быть для него удобной. Программист должен

обеспечить реакцию системы на действия пользователя именно так, как этого хочет пользователь (а не программист), иначе пользователь будет чувствовать себя беспомощным. Одна из аксиом дизайна пользовательского интерфейса такова: пользовательский интерфейс считается хорошим тогда, когда программа ведет себя в точности так, как ожидает пользователь.

Программист при проектировании диалога имеет свои представления о пользователе:

- о его целях;
- о том, как он выражает свои знания;
- о его квалификации;
- о его опыте работы с системой.

Совпадение ожиданий обоих собеседников — пользователя и программиста — сильно влияет на эффективность диалога, в частности:

- на его сжатость и лаконичность;
- на простоту понимания и простоту выражения мыслей;
- на минимизацию неопределенности при интерпретации информации, полученной от пользователя.

Ожидания пользователей могут формироваться различными путями:

- из предыдущего опыта работы;
- из разговоров с людьми («Мне один академик сказал, что это просто сделать...»);
- из литературы. (Для неспециалистов это может быть научно-популярная или фантастическая литература.)

Поэтому, приступая к разработке системы, всегда нужно знать, чего же конкретно хочет пользователь и насколько реальны его ожидания. В противном случае при сдаче системы могут обнаружиться серьезные расхождения во взглядах на ее возможности, и дальнейшая эксплуатация системы окажется под вопросом.

С учетом понятия об ожиданиях пользователя можно сформулировать три правила для программного обеспечения ЭВМ:

1. Программа не должна затруднять человека или допускать, чтобы с ее помощью человек сам создавал себе проблемы. Если в автоматизированной технологии нагрузка на пользователя только возросла по сравнению с ручной технологией, то здесь явная ошибка разработчи-

ка, связанная скорее всего с недостаточным изучением предметной области.

2. Система должна рассматривать время и данные пользователя как бесценные, а работу — священной. Поэтому она не должна тратить впустую его время или вынуждать выполнять действия сверх необходимых.

3. Программа должна защищать себя от ошибок, которые, в первую очередь, определяются здравым смыслом.

При проектировании интерфейса «человек-машина» следует иметь в виду, что система должна реагировать на сообщения и команды пользователя примерно так же, как и человек. Для оценки соответствия системы этому критерию можно предложить так называемый тест Тьюринга: может ли человек понять, с кем он общается — с человеком или машиной, — когда собеседники находятся в разных комнатах.

2.5. ПСИХОЛОГИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНТЕРФЕЙСА

Способ, которым вы выполняете какую-либо задачу с помощью какого-либо продукта, а именно совершаемые вами действия и то, что вы получаете в ответ, и является интерфейсом.

Очень многие факторы человеческой производительности не зависят от возраста, пола, образования и культуры. Эти факторы имеют непосредственное отношение к основам разработки любого интерфейса, и разработчик может облегчить себе труд, сосредоточив внимание на том, что является общим для всех людей.

Разрабатывая интерфейс в соответствии с возможностями и слабостями человека, мы помогаем пользователю не только справиться с работой, но и наполняем его жизнь позитивными эмоциями (не надо забывать, что человек управляется эмоциями!).

Философия эмоционального дизайна появилась в начале 70-х годов XX века. Впервые эту идею сформулировал японский профессор Митсуо Нагамачи (Mitsuo Nagamachi), который предложил преобразовать эмоциональные переживания и ощущения человека в конкретные свойства продукта. В начале 90-х этот подход был применен при создании уникального японского автомобиля Mazda MX-5 Miata.

Впервые в разработке автомобиля на первое место поставили не технические характеристики (скорость разгона или количество лошадиных сил), а чувства, которые испытывает водитель автомобиля — нравится ли ему звук мотора, получает ли он удовольствие от езды, от общения с машиной. Успех был ошеломляющий, и эта модель даже вошла в книгу рекордов Гиннеса как самый продаваемый автомобиль своего класса. Этот подход назвали кансей инжиниринг (kansei engineering), и он с успехом применяется во многих компаниях по всему миру.

Алан Купер, признанный мировой эксперт в вопросах создания интерфейсов, говорит, что при создании продуктов важно учитывать не только конечные цели пользователей (то, ради чего они покупают продукт), но и их **эмоциональные** цели.

Эмоциональные цели выражают то, как человек хочет себя чувствовать, работая с продуктом. Когда продукт заставляет пользователей чувствовать себя глупо или неудобно, их самоуважение и производительность труда снижаются, а недовольство растет. Стоит чуть переборщить с таким отношением к пользователям — и они воспользуются первым же шансом, чтобы избавиться от этого продукта. Такой продукт потерпит неудачу — независимо от того, насколько хорошо он позволяет пользователям достигать всех прочих целей.

Дональд Норман в результате своих исследований пришел к выводу, что для человека ценность во многом определяется не только полезным результатом, но и эмоциональным компонентом взаимодействия.

Успешный интерфейс может быть разработан на основе прагматического и эмпирического взгляда на то, что может и чего не может человеческий ум; сколько времени потребуется человеческому сознанию на выполнение тех или иных задач; какие условия повышают вероятность совершения ошибки.

Хотя у пользователей могут быть разные потребности в зависимости от конкретной задачи, в целом они имеют много общих ментальных характеристик. Перед тем, как приступать к разработке программы или пытаться учесть различия между пользователями, разработчику следует сосредоточить внимание на том, что является общим для всех людей с точки зрения требований к интерфейсу.

На первом этапе необходимо привести весь проект в соответствие с общими законами психологии, а не с так называемыми «промышленными стандартами», которые отчасти построены без учета зако-

номерностей мышления и поведения человека, например, раздвоение фокуса внимания.

Разработчики должны больше заботиться об удобстве использования, чем о внешней привлекательности продукта, тогда для пользователя все будет просто и понятно. Для этого разработчику следует развивать в себе способность чувствовать, переживать те трудности, с которыми сталкиваются его пользователи.

Пользователи не задумываются, как устроена машина, пока она справляется с их задачами. При этом им совершенно безразлично, какой процессор используется, какая операционная система, язык программирования. Для пользователя важнее всего удобство работы и решение своих задач. Но все, что он видит — это интерфейс, и с точки зрения пользователя именно интерфейс является продуктом.

Пользователь в первую очередь — человек, а уже потом эксперт или новичок. Понимание этого факта далеко не очевидно. Разработчик часто уходит от пристального рассмотрения интерфейсных проблем, ссылаясь на занятость, принятые стандарты и привычки. Но здесь таится опасность. Если пользователь не почувствует должного комфорта и уважения к себе именно как к человеку, то расценит неуважение системы как хамство, скрытую агрессию и будет соответствующим образом к ней относиться. Чтобы понять сказанное, достаточно вспомнить свои ощущения в обшарпанной и захламленной комнате и в заново обставленной комнате после ремонта. Интерфейс является ориентированным на человека, если он отвечает нуждам человека и учитывает его возможности и слабости.

Высшей степенью удобства считается ситуация, когда пользователь ощутил потребность в каком-либо ресурсе или сервисе и тут же появилась возможность эту потребность удовлетворить. Решить эту задачу без целенаправленного предпроектного обследования объекта автоматизации и глубокого погружения в технологию работы пользователя (так, чтобы можно было на равных разговаривать с ним о его работе) практически невозможно.

2.6. ПРИВЫЧКИ ПОЛЬЗОВАТЕЛЯ

При постоянном выполнении одних и тех же действий у нас формируются определенные привычки. Любая привычка означает отказ от

внимания к деталям и автоматическое выполнение действий. Поэтому задача разработчика интерфейсов — создавать интерфейсы, которые, во-первых, используют уже существующие привычки пользователей и, во-вторых, развивают у пользователя новые привычки, которые позволяют упростить ход работы.

Ломать существующие привычки и стереотипы в работе пользователя — шаг, требующий огромной ответственности и понимания объективной необходимости такого шага. Зачастую невозможно изменить привычку волевым образом. Поменяйте, например, назначение клавиш ENTER и ПРОБЕЛ, выдайте перед началом работы соответствующее предупреждение и оцените, сколько ошибок вы сделаете в процессе работы. Поэтому, если в одном из двух приложений со схожими интерфейсами произошла смена наиболее часто используемых деталей, то сформировавшиеся у пользователя привычки будут приводить к ошибкам при попытках применить старые команды. Поначалу реакция, обусловленная старой привычкой, будет приводить к ошибкам и многочисленные предупреждения, сделанные заранее, будут бесполезны.

Как бы часто мы ни говорили себе не совершать то или иное привычное действие, мы не всегда можем остановить себя. Многие проблемы, которые делают программные продукты сложными и неудобными в использовании, происходят из-за того, что в используемом интерфейсе «человек-машина» не учитываются полезные и вредные свойства человеческой способности формировать привычки. Разработчику разумнее сформировать привычки у пользователя, а потом поддерживать их, добиваясь, чтобы ментальные, осознанные операции становились автоматическими, бессознательными. Интерфейс должен предусматривать неизбежность формирования и эксплуатации привычек.

2.7. ПОДХОДЫ К ПРОЕКТИРОВАНИЮ ИНТЕРФЕЙСОВ

В процессе развития программных средств и усиления внимания к пониманию важности психоэмоционального комфорта пользователей были предложены различные подходы к созданию пользовательского интерфейса.

В первую очередь, это был инженерно-технический (Machine-Centered) подход, который рассматривает процесс разработки с точки

зрения функциональных возможностей компьютера, и когнитивный (Human-Centered) подход, рассматривающий процесс с точки зрения пользователя с позиций взаимодействия человека-оператора.

Более традиционный подход — инженерно-технический или «изнутри-вовне». При этом подходе сначала разрабатывается алгоритм, пишутся программы, затем проектируется ввод/вывод. Предполагается, что принципы решения задачи человеком подобны принципам работы компьютера. Для достижения цели задача разбивается на подцели, которые, в свою очередь, также разбиваются на более мелкие подцели.

Этот подход активно использовался в то время, когда выбор решения сильно ограничивался техническими возможностями компьютеров, и все внимание разработчиков ЭВМ концентрировалось на усовершенствовании машин, а человек мог просто использовать свою способность к адаптации для компенсации слабостей системы сопряжения «человек-машина». Поэтому в первую очередь из техники старались выжать все, что можно, а только потом уже принимались во внимание и учитывались адаптивные возможности человека, которые были достаточно высокими для компенсации недостатков сопряжения «человек-машина». Пока машина была экзотикой, особого внимания проблемам пользовательского интерфейса не уделялось.

Первые компьютеры имели недостаточный объем памяти и мощности, были дорогими и не поддавались легкой программной настройке. Поэтому с ними могли работать только инженеры и ученые-математики, которые разбирались в логике центрального процессора и мыслили в терминах машины, а значит, могли расшифровать специфические закодированные сообщения об ошибках. Таким образом, и зародилась традиция отношения программ к людям как к процессорам.

С первых же дней развития компьютерных технологий программисты приняли как аксиому утверждение, что лучший способ взаимодействия программы с пользователем — требовать от людей того же уровня совершенства, который присущ центральному процессору.

Поэтому инженерно-технический подход к созданию пользовательского интерфейса и был основан на предположении, что человек работает с компьютером подобно самому компьютеру, и, как следствие, пользователь, работающий с программой, независимо от профиля своего образования, был вынужден думать, как разработчик, а интер-

фейс, соответственно, ориентировался именно на функциональные характеристики программы.

Альтернативой инженерно-техническому подходу является когнитивный подход или подход «извне-вовнутрь». Этот подход рассматривает пользователя как центральную фигуру процесса взаимодействия с системой.

Со временем, когда технические ограничения перестали быть определяющими, появилась возможность более полно учитывать психологические особенности человека и создавать более комфортные системы, учитывающие интересы пользователя — непрофессионала в области вычислительной техники.

Появление на рынке персональных ЭВМ привело к огромному количеству таких пользователей-непрофессионалов, поэтому ожидать от этих пользователей стремления адаптироваться под технику стало бессмысленным.

Произошло также изменение отношения к компьютерам: пока машина была новшеством, люди психологически были готовы адаптироваться к ней, а теперь появились все основания считать, что машину надо приспосабливать к человеку с помощью более тщательно сконструированного интерфейса.

Разработку в таком случае естественнее проводить по пути «извне-вовнутрь». Извне находятся человек, дисплей, выводимая информация; внутри — программа, алгоритм. Этот подход ориентирован на пользователя и в настоящее время является господствующим. В общем случае сначала определяются функции будущей системы, описывается диалог пользователя, а уже потом под этот диалог пишется программа.

Когнитивный подход* рассматривает пользователя как центральную фигуру процесса взаимодействия с системой. Ориентация на характеристики пользователя, исследование когнитивных возможностей и ограничений человека позволили выявить закономерности взаимодействия человека с автоматизированной системой. Рассматривая процессы и закономерности восприятия, переработки информации и принятия решения, когнитивная психология выявила факторы, определяющие успешность выполнения задачи оператором. И это

---

* Когнитивная психология занимается изучением познавательных процессов человеческой психики.

оказались не функциональные характеристики системы, как предполагалось инженерами раньше, а качество предоставления и управления информацией с точки зрения возможностей и ограничений человека.

Однако оказалось, что анализа только процессов восприятия и переработки информации человеком недостаточно для проектирования эргономичного интерфейса, поскольку он не позволяет определить состав и последовательность выводимой на экран информации. Это привело к появлению некоторого числа методологий дизайна пользовательского интерфейса, основанных на когнитивном подходе.

2.8. ОСНОВНЫЕ МЕТОДОЛОГИИ РАЗРАБОТКИ ИНТЕРФЕЙСОВ

2.8.1. Дизайн, ориентированный на деятельность (ACD)

В этой методологии, предложенной Дональдом Норманом*, внимание уделяется прежде всего пониманию деятельности. Теория деятельности, лежащая в основе этого подхода, представляет компьютер в качестве инструмента, с помощью которого человек решает различные задачи, и именно деятельность человека влияет на интерфейс.

Согласно принципам теории деятельности (выдвинута советским психологом А.Н. Леонтьевым), человек рассматривается через призму того, как он взаимодействует с окружающим миром.

Это позволяет анализировать цели, внешние и внутренние задачи, порядок и вид операций пользователя, совершаемых для достижения итогового результата, а по результатам анализа разработать интерфейс, наиболее подходящий для данного вида деятельности.

Созданная Норманом схема (Activity-Centered Design, ACD) подчеркивает важность контекста пользователя. Метод ACD может быть полезен при разделении на составные части того, **что** делает пользователь, но не отвечает на вопрос: «Почему пользователь приступает к этой задаче?».

Строя проектирование исключительно на основе анализа деятельности и задач, мы рискуем попасть в ловушку устаревших технологий или применить модель, соответствующую целям корпорации, но не отвечающую целям пользователей.

---

* Британский психолог, вице-президент отдела высоких технологий Apple Computer.

«Если у вас возникают трудности с использованием тех или иных предметов: дверей, компьютеров или выключателей — это не ваша вина. Не корите себя. Во всем виноват дизайнер. Это вина техники или, точнее, дизайна» (Д. Норман).

2.8.2. Целеориентированный дизайн (Goal-oriented design)

В основе этой методологии разработки пользовательских интерфейсов, идеологом которой является Алан Купер, предположение о том, что тщательное изучение целей пользователя и понимание того, к чему он стремится, позволяет понять смысл деятельности для пользователя и таким образом создавать более уместные и качественные продукты.

Цели побуждают людей вести некую деятельность; понимание целей позволяет понять ожидания и устремления пользователей, что, в свою очередь, может помочь в определении видов деятельности, имеющих реальное отношение к дизайну интерфейса. Цели определяются человеческими мотивами и потому со временем не меняются или меняются весьма незначительно. Деятельность и задачи преходящи, поскольку почти целиком основаны на имеющейся на данный момент технологии.

2.8.3. Дизайн, ориентированный на пользователя (UCD)

Суть методологии сводится к изучению потребностей и возможностей конечных пользователей и адаптации продукта под их нужды. User-Centered Design (UCD) — это концепция создания программных продуктов, которыми люди хотели *бы* пользоваться.

Принципы UCD (рекомендации IBM):

- определение целевой аудитории (ЦА) — центральный этап разработки;
- понимание психологии пользователей, видение дизайна через призму взгляда представителей целевой аудитории;
- анализ информации о конкурентах — их сильные решения;
- эволюционный дизайн, построенный на обратной связи с пользователями, которая поддерживается с помощью тестирования многочисленных прототипов и управляет процессом разработки.

2.9. ОСНОВНЫЕ РЕКОМЕНДАЦИИ ПО ПРОЕКТИРОВАНИЮ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

Существует множество рекомендаций от специалистов по проектированию пользовательского интерфейса. Эти рекомендации в большей или меньшей степени применимы как к созданию настольных и/или мобильных приложений, так и к веб-разработкам. Следует обратить внимание, что эти рекомендации, сформулированные разными специалистами в различных странах, имеют много общего.

Два закона дизайна интерфейсов

Джеф Раскин, специалист по компьютерным интерфейсам, сформулировал два закона разработки пользовательских интерфейсов:

Первый закон: Компьютер не должен вредить вашей работе или своим бездействием допустить причинение вреда вашей работе.

Второй закон: Компьютер не должен тратить ваше время или требовать от вас больше работы, чем это действительно необходимо.

Три общих принципа проектирования UI

С. Жарков приводит три общих принципа проектирования пользовательских интерфейсов, которые звучат так:

1) программа должна помогать выполнить задачу, а не становиться этой задачей;

2) при работе с программой пользователь не должен ощущать себя дураком;

3) программа должна работать так, чтобы пользователь не считал компьютер дураком.

2.9.1. Восемь «золотых» правил Шнейдермана

Бен Шнейдерман, американский исследователь в области человеко-машинного взаимодействия, сформулировал восемь «золотых» правил, которые кратко можно представить в следующем виде:

1. Будьте последовательны: используйте одинаковые действия, названия, элементы управления в идентичных или похожих ситуациях.

2. Учитывайте возможности опытных пользователей: предоставьте им альтернативные способы управления программой с помощью «горячих» клавиши, макросов и т.п.

3. Используйте обратную связь: программа должна реагировать на каждое действие оператора.

4. Создавайте законченные диалоги: сформируйте последовательные действия оператора в логические группы с началом, серединой и концом. На каждом этапе поддерживайте обратную связь.

5. Используйте простые процедуры обработки ошибок: насколько возможно, спроектируйте систему так, чтобы пользователь не мог допустить серьезных ошибок, а при обнаружении ошибки предложите простые и понятные механизмы ее обработки.

6. Обеспечьте простой механизм отмены действий: такая возможность уменьшает беспокойство пользователей, т.к. они знают, что ошибочные действия могут быть отменены. Единицей обратимости может быть разовая акция, ввод данных или целая группа действий.

7. Создайте впечатление, что пользователь управляет всеми процессами: спроектируйте систему так, чтобы оператор был инициатором действий, а не ведомым.

8. Уменьшите загрузку кратковременной памяти: особенности человеческой памяти накладывают ограничения на количество, размеры и скорость чередования элементов управления.

2.9.2. Десять эвристических правил Якоба Нильсена

В 1994 году Якоб Нильсен, датский консультант по юзабилити, занимавшийся этим в фирмах IBM и Sun Microsystems, по результатам факторного анализа 249 ранее выявленных проблем юзабилити представил набор эвристик*, которые стоит учитывать при проектировании пользовательских интерфейсов.

1. Видимость состояния системы. Система должна всегда и за приемлемое время реагировать на действия пользователя и информировать его о текущем состоянии работы.

---

* Эвристика — совокупность приемов и методов, облегчающих решение практических задач.

2. Равенство между системой и реальным миром. Система должна разговаривать с пользователем на его языке, используя слова, фразы и концепции, которые уже известны пользователю. Представление информации должно быть организовано в естественном и логичном порядке.

3. Свобода действий пользователя. Пользователь должен иметь контроль над системой и возможность изменить текущее состояние программы путем отмены или повтора операций (Undo & Redo).

4. Последовательность и стандарты. Принцип последовательности означает использование одних и тех же понятий и средств для отражения схожих образов и выполнения однотипных действий. Легче всего это достигается путем использования типовых для конкретной платформы рекомендаций и соглашений.

5. Предупреждение ошибок. Система должна быть разработана так, чтобы минимизировать число ситуаций, в которых пользователь может совершить ошибку. Это лучше, чем самое информативное сообщение о возникшей проблеме. Как известно, болезнь легче предупредить, чем лечить.

6. Понимание лучше, чем запоминание. Все объекты, функции, действия должны быть видны пользователю. Он не должен запоминать и удерживать в памяти информацию из одной части диалога, чтобы применить ее в другой. В любой момент пользователю должно быть ясно, что нужно делать в данный момент. В случае необходимости пользователь должен иметь простой доступ к контекстной справке.

7. Гибкость и эффективность использования. Чтобы интерфейс программы был одинаково удобен как для новичков, так и для опытных пользователей, необходимо обеспечить альтернативные способы работы с ним. «Горячие» клавиши, тулбары, контекстные меню и т.п. — пусть пользователь сам выберет то, что ему удобнее.

8. Эстетичный и минималистичный дизайн. Диалоги не должны содержать нерелевантную или редко используемую информацию. Каждый дополнительный элемент интерфейса конкурирует с другими и отвлекает часть внимания пользователя, тем самым уменьшая относительную видимость действительно необходимой информации.

9. Распознавание и исправление ошибок. «Помогайте пользователю распознавать, диагностировать и исправлять ошибки» — говорит

Якоб Нильсен и поясняет, что сообщения об ошибках должны быть выражены простым языком (без кодов), точно описывать проблему и предлагать конструктивное решение для нее.

10. **Справка и документация.** Лучшая система та, которая может быть использована без какой-либо документации. Это идеал, но в реальности программа должна содержать необходимую справочную информацию и документацию. Любая (справочная) информация должна быть доступна для поиска, сфокусирована на задачах пользователя, последовательна в описании его действий и при этом должна быть не слишком громоздкой.

2.9.3. Принципы Usage Centered Design

Ларри Константин, идеолог концепции дизайна, ориентированного на использование Usage Centered Design (не путать с User-Centered design), представил следующие принципы разработки интерактивных систем:

1. **Структурный принцип:** Проектирование интерфейса должно вестись целенаправленно, с использованием конструктивных решений, основанных на четких и последовательных моделях, узнаваемых для пользователя. Структура интерфейса может формироваться путем группировки связанных объектов и разделения несвязанных, подчеркиванием различий между разнородными элементами и наделением похожими чертами родственных объектов.
2. **Принцип простоты:** Дизайн должен быть простым, общие задачи должны быть понятными, общение между программой и человеком должно происходить на родном для него языке.
3. **Принцип видимости:** Все необходимые для решения конкретной задачи элементы интерфейса должны быть видимыми и не отвлекать пользователя посторонней или избыточной информацией.
4. **Принцип обратной связи:** Дизайн должен информировать пользователей о выполняемых действиях, изменениях состояния или условий, об ошибках или исключениях. Эта информация должна быть актуальна и интересна пользователю и представлена в четкой, компактной и однозначной (недвусмысленной) форме.

5. **Принцип толерантности:** Дизайн должен быть гибким и терпимым к действиям пользователей, позволять отмену и повторное выполнение операций, а также предотвращать ошибки (где это возможно), интерпретируя все входные последовательности в разумные действия.

6. **Принцип повторного использования:** Интерфейс должен использовать согласованные внутренние и внешние компоненты, тем самым уменьшая для пользователей необходимость переосмысления или запоминания их (компонентов) назначения и поведения.

2.10. ПРОЦЕСС РАЗРАБОТКИ ИНТЕРФЕЙСА

Для создания диалоговых систем существует определенная последовательность действий, приводящая к достижению цели.

1. Предпроектное обследование: выяснение целей и ограничений проекта. Процесс создания интерфейса следует начать с определения целей проекта, а также выяснения внешнего и внутреннего климата предприятия/подразделения, в котором вам придется работать. Нужно детально изучить существующую и ожидаемую технологию работы. В зависимости от сложности объекта автоматизации процесс предпроектного обследования занимает от двух месяцев и более.

При обследовании нужно выяснить:

- что послужило причиной создания проекта;
- этапы создания проекта;
- какие результаты должны быть получены;
- какая информация необходима, кому и когда;
- характеристики технических устройств: размер экрана, сенсорный или нет, разрешение экрана, цветовая палитра и т.д.;
- контингент пользователей: категории пользователей, их квалификация, важность и размер групп пользователей, их мотивы, сценарии работы.

Приступая к разработке, нужно постараться как можно раньше определить **что** именно должен делать пользователь для получения того или иного результата и **как** система должна отвечать на каждое действие пользователя.

2. Концептуальный дизайн. На этой фазе разработки нужно определить какой интерфейс будет лучше подходить для достижения целей:

текстовый, графический, мультимедиа. Для этого нужно опросить достаточно большое количество будущих пользователей и выяснить, чего они ожидают от программы и как бы они хотели общаться с ней. Большинство здравомыслящих людей будет склоняться к какому-либо одному варианту, который будет соответствовать модели пользователя. Если программист ошибется в понимании модели пользователя, то созданную им систему пользователи могут отторгнуть.

Затем создается концептуальная модель программы — схема, которая показывает главные элементы, процессы и связи в программе с точки зрения пользователя. Это тот минимум, который должен знать пользователь о программе, чтобы успешно ориентироваться в ней. Элементы дизайна — окна, меню, текст — должны помочь получению этих знаний.

На данном этапе создается общая схема работы, где разработчик показывает, как он планирует распределить функции и задачи между окнами программы. Здесь же определяются способы перемещения пользователя как внутри отдельной задачи, так и между задачами; можно ли, не завершив одну задачу, начать другую.

Составленную схему работы пользователя в новой системе нужно показать пользователям и обсудить с ними те трудности, которые они обнаружат.

3. Визуальный дизайн. После уточнения можно приступать к детальной проработке каждого окна на основе четкого и точного знания задач пользователя. Создав окно, его нужно обязательно показать пользователю и выслушать его замечания. Элементы визуального дизайна должны показать пользователю для чего нужно каждое окно и как им пользоваться.

Хорошо выполненный дизайн выглядит чистым, простым и аккуратным. Его можно понять одним взглядом. Пользователь сразу должен распознать, какие данные можно редактировать, какие нет, по каким объектам можно щелкать мышью, какие объекты можно перетаскивать. Размер, цвет, яркость, местоположение, форма и текстура — это те средства, которые используются, чтобы сгруппировать элементы вместе по их важности или похожести.

Сделав свою версию интерфейса (внешнюю оболочку), попросить наиболее доброжелательных пользователей попытаться выполнить свои

задания. Задача разработчика — наблюдать за их действиями с целью выяснения ожиданий от программы и способов взаимодействия с ней.

4. Тестирование интерфейса на пользователях. Когда программа написана, необходимо протестировать интерфейс. Пяти-шести пользователей из одной категории будет достаточно. Только не надо им всем раньше времени рассказывать как все работает. Попросите их «думать вслух», озвучить свои желания и намерения; задавайте вопросы, пытаясь уточнить какова их пользовательская модель, но не подсказывайте.

Тестирование интерфейса на пользователях — это не обсуждение дизайна. В тестировании предлагается пользователю выполнить какое-либо задание самостоятельно. Задача разработчика — наблюдать и понять, что должен дать интерфейс для обеспечения удобства.

Указанные выше этапы повторяются многократно в процессе разработки программы. В результате пользователь привлекается к разработке интерфейса уже на ранних стадиях проекта, сам участвует в работе, своевременно вносит коррективы и в конце концов получает именно то, что ему нужно. Хорошие результаты дает включение кого-либо из пользователей в группу разработчиков, причем желательно, чтобы этот человек был экспертом в своей области.

Разработчику следует помнить, что бóльшая часть трудностей, связанных с использованием компьютеров, возникает в основном из-за сложности интерфейса, а не самой задачи, или же по причине недостатка старания либо сообразительности у пользователя.

3. МОДЕЛИ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

3.1. ПАРАДИГМЫ ИНТЕРФЕЙСА

Парадигма — исходная концептуальная схема, модель постановки проблем и их решения, методы исследования, господствующие в течение определенного исторического периода в научном сообществе.

В визуальном и концептуальном проектировании пользовательских интерфейсов существуют три основных парадигмы: парадигма реализации, метафорическая, идиоматическая.

Интерфейсы, ориентированные на реализации, ставят перед пользователем непростую задачу, так как опираются на *понимание* того, как работает продукт, и тем самым неявно подразумевают, что пользователь должен мыслить как разработчик и быть программистом. Это совершенно проигрышный подход, так как добрая половина пользователей имеет гуманитарный склад мышления.

Метафоры удобны в общении между людьми, однако они малопригодны для проектирования программных продуктов и по этой причине препятствуют созданию действительно хороших интерфейсов.

Метафорические интерфейсы основаны на *интуитивных* представлениях о работе продукта и являются рискованным подходом, так как использование метафор сопряжено с серьезными трудностями при построении аналогий между обычной реальностью и понятиями IT-технологий.

Идиоматические интерфейсы (идиома — словосочетание, смысл которого не вытекает из смысла входящих в него слов, например: лапшу на уши вешать, втирать очки) основаны на *обучении* пользователя тому, как достичь результата, что является более естественным процессом для людей. Идиому не нужно стараться понять рассудком или интуитивно — ее надо выучить.

Разработка пользовательских интерфейсов как дисциплина прошла путь от сильной концентрации на технологии (парадигма реализации)

до столь же выраженной концентрации на метафоре, хотя в современных интерфейсах просматриваются все три парадигмы.

Существует три модели пользовательского интерфейса: модель реализации; модель пользователя; модель представления.

3.2. МОДЕЛЬ РЕАЛИЗАЦИИ

Самый важный и принципиальный вопрос, на который должен ответить разработчик: чьи интересы важнее — пользователя или разработчика?

Что будет делать разработчик: приближать программные продукты к образу мышления пользователей или заставлять пользователей подстраиваться под чуждую им компьютерную логику, облегчая себе жизнь?

Представление о том, как в действительности работает программа, называется *моделью реализации*, поскольку такая модель описывает подробности реализации программы в коде. Очевидно, что, кроме программиста, никто лучше не ориентируется в этой модели. Эти интерфейсы выражены в терминах, отражающих конструкцию продукта — на каждый модуль кода выводится диалоговое окно, каждая функция из технического задания реализована пунктом меню или отдельной кнопкой, команды и процессы являются точным отражением внутренних структур данных и алгоритмов.

Чтобы успешно взаимодействовать с таким интерфейсом, пользователь должен понимать, как работает программа. Следование парадигме реализации означает, что проектирование пользовательского интерфейса целиком основано на модели реализации.

Интерфейс, созданный на основе модели реализации, может функционировать только обладая знанием о том, как работает программа.

Чтобы пользоваться таким интерфейсом, пользователь должен разобраться в том, как программа устроена.

Интерфейсы, ориентированные на реализацию, строить проще всего: Каждый раз при создании функции программист просто добавляет фрагмент интерфейса, реализующий эту функцию. Такой интерфейс легко отлаживать, а когда что-то работает не так, легко выяснить, в чем причина.

Парадигма реализации вполне устраивает инженеров, так как помогает понять, что происходит внутри программы, и они готовы на это тратить силы и время. Пользователи не имеют ни малейшего желания разбираться в интерфейсе, и успех в решении своих задач им важнее, чем понимание устройства программы.

Тот факт, что подобные элементы неоправданно усложняют жизнь пользователей, кажется разработчикам несущественным побочным эффектом.

Модель реализации (программиста) — самая простая для отображения, поскольку может быть описана просто и однозначно в виде спецификации на программу.

Пользовательские интерфейсы, спроектированные программистами, которые досконально знают принципы работы программы, имеют модель представления, очень точно следующую модели реализации. Программисты больше разбираются в функциях, а не в интерфейсе. От программиста не требуется умение предоставлять пользователю метафоры пользовательского интерфейса, это основная задача проектировщика интерфейса (к России это почти не относится, у нас все делает программист.) С точки зрения разработчика, такая модель логична, достоверна и точна; но, к сожалению, она малопонятна и неэффективна для пользователя.

Очень часто к таким решениям приводят «сжатые» сроки разработки — времени всегда не хватает — и чтобы отчитаться по формальностям, удобно каждую функцию поместить в отдельном окне, показать функциональность заказчику и победно завершить разработку. Но устроит ли такое решение конечного пользователя?

Следование модели реализации в многооконном диалоге приводит к плохому интерфейсу.

3.3. МОДЕЛЬ ПОЛЬЗОВАТЕЛЯ

Собственное понимание пользователем того, что делает программа и как с ней общаться, — это *модель пользователя*.

Упрощенную мысленную схему, достаточную для описания взаимодействия с программой, но не всегда точно отражающую ее внутреннее

устройство, называют *ментальной моделью* или *концептуальной моделью пользователя*.

Интерфейс, следующий ментальной модели пользователя, существенно выигрывает по сравнению с интерфейсом, отражающим модель реализации, поэтому единственный успешный вариант интерфейса — это реализующий пользовательскую ментальную модель.

Мы можем знать или не знать, как, например, сотовый телефон находит абонента, как происходит шифрование данных при пересылке сообщений, где и как хранятся данные о контактах. Все это соответствует модели реализации. Эти знания совершенно не помогают нам понять, как пользоваться телефоном.

В программных приложениях с высокой сложностью реализации связи между действиями пользователя и реакцией программы часто лишены аналогий с привычным миром, поэтому для большинства людей они остаются непонятными, а модель пользователя оказывается неполной и не всегда адекватной.

3.4. МОДЕЛЬ ПРЕДСТАВЛЕНИЯ

Кроме модели реализации, подвластной программисту, существует еще *модель представления* — избранный проектировщиком интерфейса способ предъявления пользователю функционирования программы (модели реализации).

Модель представления (она же — модель проектировщика интерфейса) является промежуточным звеном между моделью пользователя и программиста, поскольку разработчики обычно не контактируют с конечными пользователями, для которых разрабатывают программы, в силу ряда объективных и субъективных причин.

В отличие от моделей пользователя и программиста, модель представления является тем аспектом программного обеспечения, который в значительной степени поддается контролю со стороны проектировщиков.

Одна из важнейших целей проектировщика интерфейса — максимально приблизить модель представления к ментальной модели пользователей. Таким образом, проектировщикам очень важно де-

тально знать, как целевые пользователи видят работу, выполняемую с помощью программы.

В программном мире модель представления программы может сильно отличаться от модели реализации. Например, накопителем на жестком диске может быть локальный или сетевой диск или вообще удаленный файловый сервер, а для пользователя это будет представляться одним значком.

Модель представления упрощает понимание модели реализации, и чем ближе модель представления к пользовательской ментальной модели, тем легче пользователю работать с программой и понимать ее.

Но в большинстве случаев программисты облегчают себе жизнь, приближая модель представления к модели реализации, совершенно не заботясь о том, что это приближение сильно отличается от ментальной модели пользователя, которую он ожидал получить.

4. МЕТАФОРЫ И ИДИОМЫ

Метафорические интерфейсы

Говоря о метафорах в контексте пользовательского интерфейса и проектирования взаимодействия, мы подразумеваем визуальные метафоры — картинки, которые служат представлением атрибутов или назначения объектов.

Пользователи воспринимают передаваемый метафорой образ и предположительно могут расширить его до понимания предназначения объекта. Метафоры могут охватывать широчайший диапазон: от крохотных пиктограмм на панели инструментов до полноэкранных изображений.

Мы воспринимаем метафоры интуитивно, действуя на основе предположений. Мы видим связь между разными объектами и обращаем внимание на их сходство, не отвлекаясь на различия. Наш мозг схватывает значения метафорических элементов интерфейса, поскольку мысленно связывает их с понятиями, усвоенными ранее.

Метафорический интерфейс полагается на особенности стиля мышления пользователей, которые могут не знать языка, не обладать знаниями или не способны установить связи между метафорой и обозначаемой сущностью.

Ограничения метафор

Метафоры широко распространены в реализациях интерфейсов, однако они не являются надежным фундаментом для проектирования пользовательского интерфейса.

Метафоры имеют множество ограничений, когда речь идет о современных информационных системах.

1. Метафоры плохо масштабируются. Метафора, уместная для обозначения простого процесса в маленькой программе, часто дает сбой, когда процесс усложняется. Крупные пиктограммы файлов были хорошей идеей во времена, когда в компьютерах использовались дискеты и 10-мегабайтные жесткие диски с парой сотен файлов. Сейчас, когда

на дисках объемом сотни гигабайт хранятся десятки тысяч файлов, подобные пиктограммы стали слишком громоздкими для эффективной работы.

2. Метафоры предполагают сходство ассоциативного мышления у разработчика и пользователя. Если пользователь и проектировщик относятся к разным культурам, метафора может и не работать.

3. Непонимание может возникнуть даже между представителями близких культур или в рамках одной культуры. Что, например, означает изображение самолета — «расписание вылетов и прибытий самолетов» или «заказ авиабилетов»?

4. Хотя метафора предлагает новичку способ быстрее освоиться с интерфейсом, она превращается в непосильную обузу, когда бывший новичок переходит в середняки. Отражая физический мир механизмов, большинство метафор концептуально загоняют пользователей в узкие рамки, навсегда ограничивая возможности программного продукта.

5. Метафоры трудно подбирать. Нетрудно найти визуальные метафоры для физических объектов вроде принтеров и документов. Гораздо труднее или вообще невозможно подобрать метафору для процессов, отношений, служб и преобразований понятий, чаще всего ассоциируемых с программными продуктами.

Весьма проблематично найти визуальную метафору для переключения каналов, приобретения вещи, поиска литературы, задания формата, изменения разрешения изображения и выполнения статистического анализа, однако именно такие операции чаще всего выполняются с использованием программ.

6. Метафоры ограничивают наше мышление. Самой же значимой проблемой, возникающей в связи с метафорами, является тот факт, что они привязывают интерфейсы к объектам механической эры.

С расширением метафор связаны две ловушки — одна для пользователя, другая для проектировщика.

Проектировщик испытывает искушение представить программный продукт в терминах метафоры, чтобы соответствовать ожиданиям пользователя. Однако перенос механических процессов в компьютер обычно ухудшает их.

Например, «папка» в современных компьютерных системах. В мире бумаги один документ не может находиться сразу в двух папках в кар-

тотечном шкафу согласно единой схеме, принятой в организации — по номеру или алфавиту. Цифровые продукты не имеют таких ограничений, но слепое стремление к метафоре в интерфейсе резко ограничивает нашу способность хранить документ согласно сразу нескольким схемам организации одновременно.

Идиоматические интерфейсы

Идиоматическое проектирование, которое Тед Нельсон* (Ted Nelson) назвал «проектированием принципов», основано на том, как человек изучает и применяет идиомы и речевые обороты, такие как «свой в доску» или «тертый калач».

Идиоматические пользовательские интерфейсы решают проблемы, с которыми не справляются предыдущие два типа интерфейсов, поскольку сосредоточиваются не на технических знаниях и не на интуитивных представлениях о функциях, а на изучении простых, неметафорических визуальных и поведенческих идиом, необходимых пользователю для достижения целей и решения задач.

Наш мозг обладает удивительной способностью к изучению и запоминанию большого количества идиом, причем делает это легко, обходясь без параллелей со знакомыми ситуациями и без понимания «внутреннего устройства» идиомы.

Графические интерфейсы — в основном идиоматические

Большинство элементов интуитивно понятного графического интерфейса в действительности представляют собой визуальные идиомы. Окна, заголовки, кнопки закрытия, разделители экрана, гиперссылки и раскрывающиеся списки — все эти объекты мы впервые узнали как идиомы, а не постигли интуитивно, как метафоры.

Мышь как устройство ввода отнюдь не метафорична. Во внешности мыши нет ничего, что указывало бы на ее предназначение или способ применения, и она не похожа ни на что из нашего прошлого опыта, так что постижение этого устройства не опирается на интуицию. Более того, мышь похожа на микрофон из лингафонного кабинета. Тем не менее, достаточно один раз показать человеку, как ею пользоваться, и он на

---

* Американский социолог, философ и первооткрыватель в области информационных технологий.

всю жизнь это запомнит. Нам не нужно знать, как мышь устроена, но любой человек быстро освоит работу с ней. Это и есть идиоматическое освоение.

Многие знакомые нам элементы графического пользовательского интерфейса, часто принимаемые за метафоры, на деле являются идиомами. Окна с изменяемым размером и бесконечно вложенные файловые каталоги не метафоричны, поскольку у них нет аналогов в реальном мире. Вся их сила как раз в простоте идиоматического освоения.

Хорошую идиому достаточно изучить только один раз

Мы склонны думать, что изучать интерфейсы трудно, поскольку основываемся на опыте работы с интерфейсами, созданными по модели реализации. Эти интерфейсы очень сложны в изучении, потому что эффективная работа с ними требует от пользователя понимания как работает программа.

Большая часть того, что мы знаем, усвоена нами без понимания: мелодии, названия и изображения брендов, расположение комнат и мебели в нашем доме. Мы не понимаем, почему фигура человека выглядит так, а не иначе; но мы просто знаем, что перед нами человек. Мы отличаем его от других, потому что посмотрели на него и автоматически запомнили.

Главное наблюдение, связанное с идиомами, состоит в том, что, хотя их приходится изучать, делать это весьма просто, а хорошие идиомы достаточно изучить один раз. Человеческий мозг может осваивать идиомы интерфейса — выключатели, кнопки закрытия окон, раскрывающиеся меню и комбо-списки с первого раза.

Идиоматическое проектирование — это будущее проектирования взаимодействия. Следуя этой парадигме, мы опираемся на естественную человеческую способность к обучению и не вынуждаем пользователя разбираться в том, как работает программа. Метафоры предлагают новичкам небольшой выигрыш при обучении, зато впоследствии оборачиваются заметными затратами. Лучше всего проектировать на основе идиом, применяя лишь самые мощные и подходящие метафоры.

Если найдутся метафоры, то их нужно использовать, но нельзя подгонять свой интерфейс под произвольный метафорический стандарт.

5. КРИТЕРИИ ХОРОШЕГО ДИАЛОГА

Поскольку при построении диалога «человек-компьютер» машина не может полностью сыграть роль человека, то разработчику не следует слишком сильно ориентироваться на модели общения людей друг с другом. Однако, как правило, пригодность структуры диалога можно оценить путем анализа того, какая структура диалога «человек-человек» используется в аналогичной ситуации.

Чтобы сделать эту оценку более точной, воспользуемся в первую очередь пятью основными критериями хорошего диалога:

– естественность;
– последовательность;
– краткость;
– поддержка пользователя;
– гибкость/тактичность.

Рассмотрим эти критерии более подробно.

5.1. ЕСТЕСТВЕННОСТЬ

Естественный диалог — это такой диалог, который не вынуждает пользователя, взаимодействующего с системой, существенно изменять свои традиционные способы решения задачи. Определяется насколько автоматизированная технология адекватна существовавшей ранее ручной. Конечно, полностью сохранить прежнюю технологию не удастся по объективным соображениям — поменялись инструментальные средства, но воплотить основные этапы и метафоры можно.

Естественный диалог максимально поддерживает привычки пользователя. Сказанное относится и к смене версий программы. Новая версия должна унаследовать основные черты интерфейса для облегчения адаптации пользователя. К сожалению, этого нельзя сказать о продуктах Microsoft. Например, при смене Office 2003 на Office 2007 или Windows XP на Windows 8 или Windows10 — интерфейс поменялся на 90% и стал далеко неочевидным.

Диалог должен вестись на родном языке пользователя. Стиль диалога должен быть разговорным, а не письменным. Однако следует избегать как чрезмерной напыщенности, так и фамильярности. Добавление имени пользователя к вопросу, задаваемому системой, — прием, который очень быстро надоедает даже неискушенному пользователю. Поэтому краткая подсказка в виде: «Вариант?_« более информативна, чем вежливая фраза «Николай Петрович, пожалуйста, введите нужный вариант_«. Еще лучше обеспечить пользователю прямой выбор пункта меню с помощью мыши или клавиатуры.

Имя пользователя имеет смысл выводить в момент приветствия системой пользователя после ввода пароля. Пользователь должен получить подтверждение, что он правильно идентифицирован системой и получил свои полномочия. В течение сеанса имя пользователя должно высвечиваться на экране, например в заголовке окна или в статусной строке.

Фразы, по возможности, не должны требовать дополнительных пояснений. Слова ПЕЧАТЬ, КОПИРОВАТЬ, ПЕРЕМЕСТИТЬ имеют очевидное значение, в то время как команды PIP (копирование файла в первой ОС для персональных ЭВМ CP/M) или MV (перемещение файла в ОС UNIX) не очевидны.

Употребление жаргона вполне допустимо, но при условии, что он используется в среде пользователей, а не специалистов по вычислительной технике. Например, разработчик может назвать режим работы: «Коррекция файла поступлений», но, если пользователь называет ее «Обработка ведомости У25», то именно так и надо называть решение этой задачи. Некоторые фразы могут иметь ярко выраженную эмоциональную окраску.

Другим важным аспектом естественности является тот порядок, в котором система запрашивает информацию. Всегда следует придерживаться того порядка, в котором пользователь обычно обрабатывал информацию в ручной технологии, а не предлагать тот, который удобен программисту и соответствует модели реализации.

Не следует заставлять пользователя вручную обрабатывать данные перед вводом в систему или после вывода из системы, облегчая жизнь компьютеру или программисту. К тому же это снижает надежность системы.

Негативный пример: в одном из отечественных учебных заведений преподаватель ежемесячно составляет Web-отчет о проведенных занятиях, хотя в бюро расписаний все известно, а в конце года он вручную суммирует свою ежемесячную нагрузку для сравнения с плановой и фактической. Разработчик, желая облегчить себе жизнь, должен избегать бессмысленного с пользовательской точки зрения порядка постановки вопросов.

Неестественный диалог часто является следствием того, что разработчик незнаком с теми способами, которыми пользователь обычно решает задачу без поддержки системы. Если в процессе эксплуатации налицо дополнительная работа по ручной обработке данных перед вводом или после вывода либо пользователю требуется карандаш и бумага для сохранения результатов — это признак плохо спроектированной системы и диалога. Две технологии могут сосуществовать только на этапе замены ручной технологии автоматизированной, пока пользователи не начнут доверять системе.

Необходимо разрабатывать программы так, чтобы они были визуально понятны пользователю и была возможность напрямую манипулировать объектами. Для наглядности можно использовать различные метафоры (папка, корзина для мусора, рабочий стол), которые поддерживают концепцию узнавания. Пользователям естественнее воспринимать осмысленные ассоциации, чем команды.

5.2. ПОСЛЕДОВАТЕЛЬНОСТЬ/СТАНДАРТИЗАЦИЯ

Диалог, отличающийся логической последовательностью, гарантирует, что пользователь, освоивший работу одной части системы, не запутается, разбираясь с особенностями другой части системы (наследование идеологии или согласованность регулярно повторяющихся действий). Если приложение взаимодействует с пользователем единообразно, то он может больше времени посвятить выполнению своей непосредственной работы, а не исследованию и освоению нюансов интерфейса.

В многозадачных средах типа WINDOWS разрешается многим задачам делить экран. Это становится возможным благодаря тому, что каждая программа взаимодействует с некоторой частью экрана,

называемой окном. Окно, с которым пользователь общается в данный момент, называется активным.

Программное обеспечение имеет четыре элемента, описывающих пользовательский интерфейс:

1. Представление (Presentation) — **что́** пользователь видит.
2. Взаимодействие (Interaction) — **как** пользователь взаимодействует с компонентами.
3. Действия (Actions) — каким образом выполняются сходные операции.
4. Последовательность действий (Process Sequence) — как пользователь и компьютер взаимодействуют (подробной информации нет).

Рассмотрим эти элементы несколько подробнее.

1. Представление — это наглядный вид интерфейса, то, что пользователь видит на экране. Прикладные программы предоставляют пользователю два вида информации: объекты и действия. Объект — это основной термин того, чем пользователь может манипулировать и на чем сосредоточено его внимание. Действия — способ, которым можно пользоваться для создания, изменения и управления объектами. Действия меняют свойства объектов. Например, объект — фрагмент текста, свойство — размер шрифта, действие — способ, которым можно поменять размер шрифта.

Основными компонентами пользовательской среды являются экран, окна, пиктограммы, курсор/указатель мыши и выпадающие меню. Отсюда — аббревиатура WIMP: Windows, Icons, Mouse, Pull down menu. В окнах помещаются объекты, которыми управляет пользователь; пиктограммы изображают объекты, которые доступны, но временно не используются; указатель мыши — курсор, который, двигаясь внутри окна/экрана, позволяет пользователю выбрать любое действие; выпадающее меню необходимо для выбора действий из списка возможных.

2. Взаимодействие — описывает механизм, с помощью которого пользователь работает с компонентами интерфейса. В интерфейсе, управляемом пользователем, фундаментальной является концепция УКАЖИ и ВЫБЕРИ (Point-and-Select), т.е. пользователь указывает и выбирает то, что он хочет. Управление при этом осуществляется как с клавиатуры, так и с помощью мыши, причем при использовании

клавиатуры действия пользователя ограничены активным окном, а с помощью мыши пользователь может указать любой видимый объект на экране.

3. Действия. Согласованность регулярно повторяющихся действий — важное звено концептуальной пользовательской модели. Например, текстовый и графический редакторы — это совершенно разные программы, разработанные по определенным правилам. Однако и та и другая включает функции создания и редактирования файлов, обе должны уметь загружать и сохранять файлы.

4. Последовательность в построении фраз предполагает, что вводимые коды, например ключевые слова, всегда трактуются одинаково. Диалог не должен содержать противоречий. Например, в разных ветвях диалога команда LIST означает вывод данных на экран, а иногда на принтер.

Стандартные ответы или реакции системы должны быть действительно стандартными. Если пользователь для ответа на какой-либо вопрос может получить помощь, нажав клавишу F1, то при ответе на любой другой вопрос он должен получать помощь путем нажатия клавиши F1. В идеальном случае клавиша F1 не должна иметь других функций, кроме функции вызова справочной информации или помощи системы. Аналогично клавиатурная комбинация Ctrl-C во всех приложениях Windows означает копирование объекта или фрагмента.

Последовательность в использовании формата данных означает, что аналогичные поля будут всегда представляться системой в одном и том же формате. Если поле для работы с датой имеет формат ДД/ММ/ГГ, то все аналогичные поля должны иметь такой же формат. Если пользователи могут сокращать входные данные в одном случае, то они должны иметь возможность использовать такие же сокращения и в других аналогичных случаях.

Последовательность в размещении данных на экране в разных ситуациях, сходных по реализуемым функциям, является гарантией того, что пользователю известно, где искать на экране инструкции, сообщения об ошибках и т.д. Это уменьшает время сканирования экрана и, соответственно, нагрузку на глаза. Нужно соблюдать также последовательность и при выделении информации так, чтобы, например, появление на экране участка в инверсном изображении или ярком цвете всегда

указывало на важность сообщения. В таких случаях нелишне бывает добавить и звуковой сигнал для привлечения внимания пользователя.

Для обеспечения принципа последовательности диалога общесистемные операции, такие как открытие или сохранение файла, рекомендуется реализовывать через стандартные диалоговые окна.

5.3. КРАТКОСТЬ

Краткий диалог требует от пользователя ввода только минимума информации, необходимой для работы системы. Взаимодействие с кратким диалогом более быстрое и осуществляется с меньшим числом ошибок, поскольку напрямую зависит от количества требуемых нажатий на клавиши.

В диалоге не следует запрашивать информацию, которая может быть сформирована автоматически или уже была введена ранее, например, количество отработанных часов за месяц. Если информация один раз попала в систему — система ее знает и больше никогда об этом не спрашивает пользователя — введенная информация священна. Точно так же не стоит требовать ввода информации, которая не используется системой.

Полезным способом сокращения вводимых данных является ввод значений по умолчанию. Но предлагаемые системой значения должны быть представлены пользователю для получения подтверждения.

Метка окна ввода должна быть краткой, по возможности использующей минимальное количество слов: например, «Фамилия», а не «Введите здесь фамилию клиента».

Вся информация не должна выводиться сразу же только потому, что она доступна системе. Выходные данные должны содержать именно ту информацию, которая сейчас требуется пользователю. Полезно следовать принципу последовательности раскрытия данных. Например, на экране показывается только название предприятия и его адрес, но для детализации пользователь может нажать кнопку, чтобы увидеть профиль предприятия, филиалы, фамилии директоров с их телефонами. Информация выводится в приемлемой форме, с привлечением минимума средств для выделения информации (чтобы пестрый экран не утомлял глаза).

5.4. ПОДДЕРЖКА ПОЛЬЗОВАТЕЛЕЙ

Поддержка пользователя в процессе диалога — это мера помощи, которую диалог оказывает пользователю при работе с системой. Основными аспектами этой поддержки являются количество и характер имеющихся инструкций; характер выдаваемых сообщений об ошибках; подтверждение каких-либо действий системы.

Инструкции для пользователя выводятся в виде подсказок либо справочной информации. Характер и количество инструкций должны соответствовать опыту работы пользователя с системой и его намерениям. Нерационально выдавать инструкции, которые не требуются пользователю или не имеют отношения к его проблеме.

Справочная информация должна появляться тогда, когда она требуется, в приемлемой форме и в фокусе внимания пользователя. Для простейших справок подходят хинты (hints) — всплывающая подсказка, появляющаяся после наведения курсора на объект или команду.

Не рекомендуется использовать для подсказок пользователю о значении тех или иных элементов интерфейса строку статуса (нижняя строка окна). Если пользователь наводит курсор на элемент интерфейса — нельзя давать в строке статуса его краткое описание. В этот момент происходит раздвоение фокуса внимания, что создает дискомфорт. В этом случае читать сообщение можно либо переведя взгляд в другое место экрана, либо напрягая периферийное зрение. Разумеется, читать подсказку в таких условиях мало кто будет.

На самом деле, строка статуса предназначена для двух вещей: либо это строка собственно статуса, т.е. отображает текущее состояние, либо панель инструментов для опытных пользователей. Для первого случая строка может содержать номер текущей страницы документа, навигационные данные о местоположении, включенные режимы (Вставка/Замена), индикатор степени выполнения (желательно не выносить его в другое место). Для второго случая в качестве иллюстрации можно предложить статусную строку главного окна WINDOWS: здесь есть и открытые папки, и запущенные программы, и наиболее важные кнопки панели инструментов.

Сущность диалога, поддерживающего пользователя, состоит в том, что пользователю необходимо обеспечить более или менее адекватную

поддержку в понимании своих ошибок, в контроле за состоянием запущенных процессов, в наличии средств воздействия на систему. Диалог может не удовлетворять этому критерию из-за того, что разработчик не смог правильно оценить потребность пользователя в поддержке, или потому, что средства поддержки не могут отразить всего многообразия требований пользователя.

5.5. СООБЩЕНИЯ ОБ ОШИБКАХ

Как указывает крупнейший авторитет в области юзабилити Дональд Норман, пользователи часто винят себя в ошибках, которые допустили проектировщики продукта. Если вы не получаете жалоб от пользователей, это еще не означает, что они радуются, когда получают сообщения об ошибках.

Диагностические сообщения, выдаваемые системой, и обработка ошибок — область особого внимания разработчика. Сообщения об ошибках неизбежно прерывают работу модальными диалоговыми окнами.

Модальное окно — это всплывающее окно, отображающее или запрашивающее информацию. Модальные окна могут содержать информацию, дополняющую информацию основного окна. Модальное окно поддерживает действия, которые пользователь **обязательно** должен выполнить перед закрытием окна. Модальное окно требует, чтобы пользователь закончил нужное действие перед возвратом в точку, из которой оно было вызвано (например, сохранение результатов редактирования в файле и продолжение редактирования).

Модальное окно бывает прикладным или системным. Прикладное окно позволяет пользователю работать с окнами, которые управляются другими программами, не закрывая данного заблокированного окна. Системное модальное диалоговое окно останавливает выполнение программ во **всех** окнах до тех пор, пока оно не будет закрыто.

Отображая модальные диалоговые окна, следует иметь в виду, что не рекомендуется делать их размеры слишком большими. Умеренные размеры предоставляют достаточно места для размещения диалога, и в то же время окна не занимают слишком большую долю экрана.

Как правило, ошибки возникают из-за некорректного ввода данных пользователем. Условно ошибки можно разделить на три класса:

1. Система выходит из строя. Здесь явный просчет разработчика, который не обеспечил входной контроль и устойчивость программы к различным наборам исходных данных. В результате система выполняет некорректные операции.

2. Система обнаруживает невозможность дальнейшей обработки полученных данных и просит пользователя скорректировать их. Вводится синтаксически верная, но логически противоречивая информация. Требования пользователя не осуществимы.

3. Система выполняет задачу, но результат отличается от того, который ожидал пользователь, например работа со старыми версиями файлов (отчет на за прошлый квартал).

Сообщения об ошибках имеют следующие недостатки: раздражение пользователя, внушение ему чувства вины, отвлечение его от работы и, как правило, отсутствие каких-либо инструкций о том, что делать в сложившейся ситуации, или еще хуже — принятие решения в ситуации, совершенно не знакомой для пользователя (рис. 1).

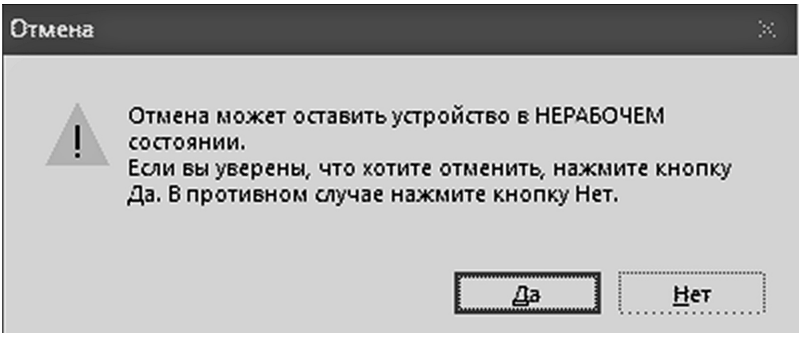

Рис. 1. Принятие решения без объяснения последствий

Сообщение об ошибке должно отвечать на три вопроса:

1. В чем заключается ошибка?
2. Как исправить проблему сейчас?
3. Как сделать так, чтобы ошибка не повторилась?

Текст сообщения, по возможности краткий, должен излагаться в терминах пользователя. Сообщение должно точно объяснять, в чем

заключается ошибка и какие действия надо предпринять для ее устранения, а не ограничиваться общими фразами типа: «Вы ошиблись», «Ошибка» или кодами ошибок «Ошибка 27» (рис. 2, 3).

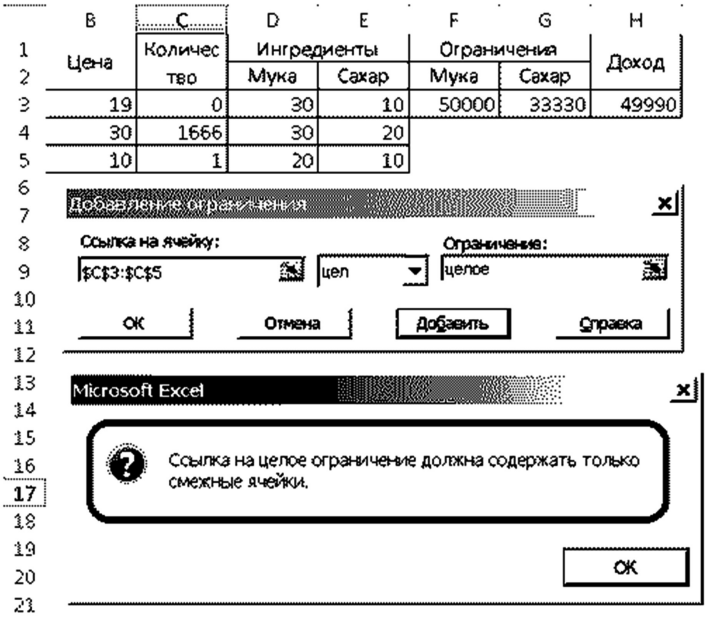

Рис. 2. Неинформативная диагностика ошибки

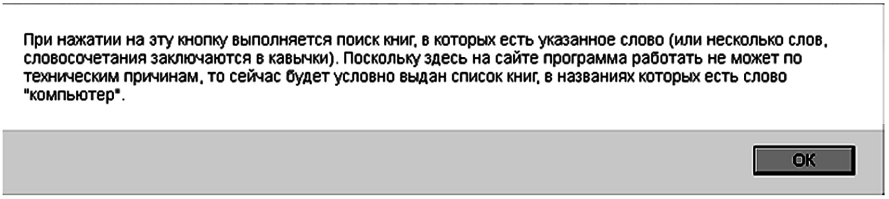

Рис. 3. Попытка разработчика оправдаться

Важно, чтобы сообщение об ошибке появлялось близко к тому месту, где сфокусировано внимание пользователя; само сообщение не должно закрывать место ошибки или ошибочные данные должны включаться в текст сообщения об ошибке. Окно с сообщением об ошибке должно всесторонне охарактеризовать проблему (рис. 4).

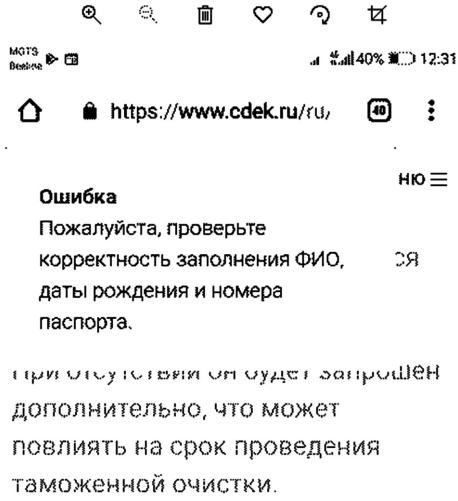

Рис. 4. Где исправлять ошибку и что именно — непонятно

Появление диагностического сообщения в другом месте раздваивает фокус внимания и создает психологический дискомфорт (рис. 5, 6). По этой причине в Windows, например, после запуска процесса песочные часы появляются там, где находился курсор.

Столкнувшись с проблемой, программа должна предоставить пользователю информацию, которая необходима для принятия решения по устранению этой проблемы. Следует четко описать проблему, возможные варианты решения, действия, которые предпримет программа по умолчанию, и сообщить, утрачены ли данные, и если да, то какие именно.

Программа обязана предложить как минимум одно решение здесь же, в диалоговом окне. Окно должно содержать кнопки, позволяющие справиться с проблемой тем или иным способом. Если, скажем, недоступен принтер, диалоговое окно должно предложить возможность подключить другой принтер или отложить печать.

Чтобы избавиться от сообщения об ошибке, нужно снизить вероятность совершения этой ошибки пользователем — это лучший способ избавиться от сообщений об ошибках. Вместо ввода текста надо перейти к выбору из списка: например, название улицы — при вводе адреса или при вводе номера телефона разрешить вводить только цифры.

Рис. 5. О какой сумме идет речь и где ее вводить непонятно, кнопка «Далее» все равно доступна. Раздвоение фокуса внимания между кнопками и сообщением об ошибке

Другой превосходный способ избавиться от сообщений об ошибках — сделать программу достаточно сообразительной, чтобы она не требовала лишнего. Многие сообщения об ошибках имеют такой вид: «Неправильный ввод. Пользователь должен ввести XXXX». Если программа знает, что именно должен ввести пользователь, почему бы ей не ввести это самой, перестав трепаться попусту? Вместо того чтобы требовать от пользователя ввести имя файла и давать ему возможность ошибаться, заставьте программу запомнить, с какими файлами он работал раньше, и выведите их список. Еще один пример: сделайте так, чтобы программа получала дату от системных часов, а не требовала ее пользователя.

Одной из проблем сообщений об ошибках является то, что они уведомляют о неприятностях по факту. Они говорят: «Случилось нечто плохое, и все, что вы можете сделать, — признать катастрофу» (рис. 7). Полезность таких сообщений сомнительна. И все эти диалоговые окна снабжены кнопкой OK, требующей от пользователя стать соучастником проблемы.

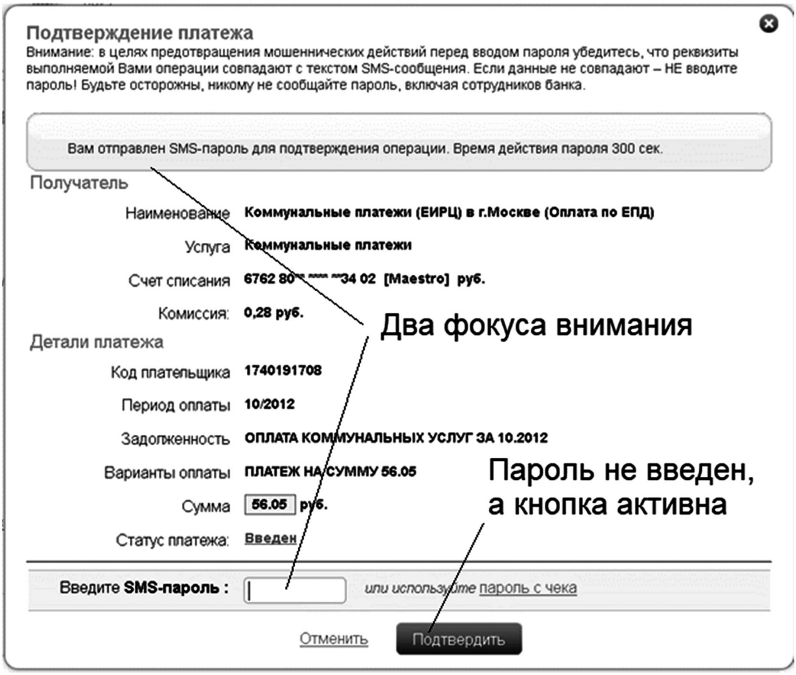

Рис. 6. Раздвоение фокуса внимания

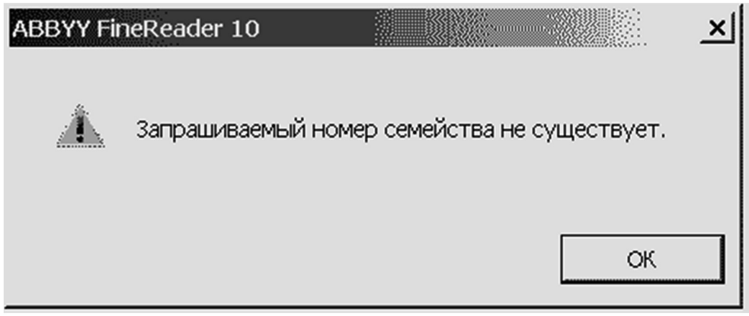

Рис. 7. Совершенно неинформативное сообщение.
Скорее всего информация для разработчика

Чтобы сделать ошибки невозможными, разработчик может пойти двумя путями: либо лишить пользователя возможности выполнить некорректную операцию в данном режиме, либо отключить ресурс,

использование которого может привести к ошибке. Например, при форматировании дисков можно заблокировать форматирование диска С, либо вообще диск С не показывать пользователю, что делает ошибку невозможной.

Уведомление сообщает пользователю о действии, предпринятом программой, а подтверждение, кроме того, дает пользователю полномочия отменить это действие, если, конечно, пользователь понимает, о чем идет речь. Идея уведомлений — в том, чтобы информировать пользователя (рис. 8, 9).

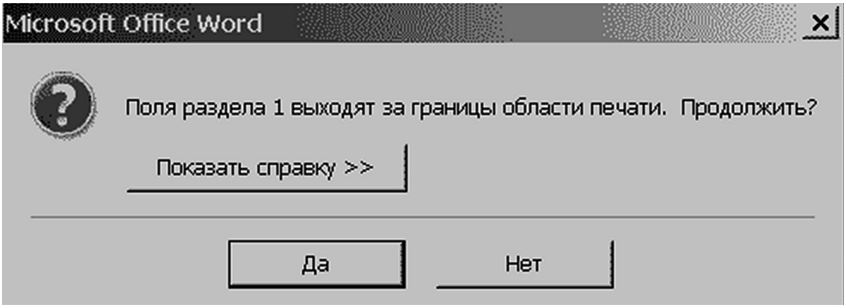

Рис. 8. Совершенно неконструктивное объяснение ситуации с попыткой переложить ответсвенность на пользователя (раздел «Показать справку >>» такой же бесполезный)

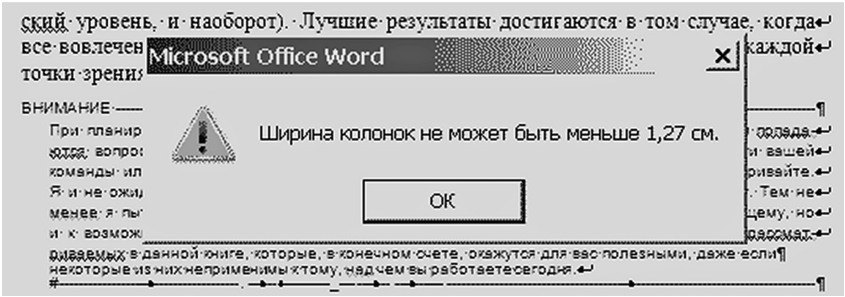

Рис. 9. Факт какой-то ошибки обнаружен, но о чем идет речь и что делать дальше — непонятно

Сообщения, подтверждающие какие-либо действия системы, требуются для того, чтобы пользователь мог еще раз убедиться, что система

выполнила или будет выполнять требуемое действие. Хотя такие сообщения существенны, использовать их следует не очень часто.

Когда приложение не уверено в своих действиях, оно нередко просит у пользователя одобрения и выдает диалоговое окно, подобное показанному на рис. 10.

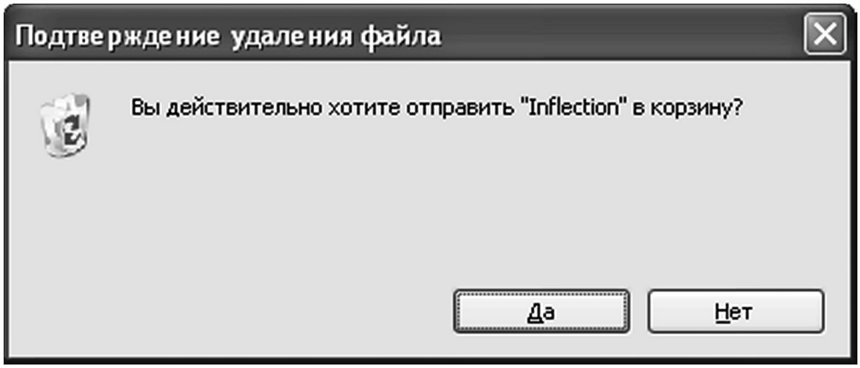

Рис. 10. Запрос на подтверждение действий

Иногда такое подтверждение предлагается пользователю потому, что программа сомневается в его действиях. В других случаях программа чувствует себя недостаточно компетентной для принятия решения и предлагает пользователю сделать выбор. Окна подтверждений всегда появляются по инициативе программы и никогда — по инициативе пользователя. Это означает, что они часто не связаны с целями пользователя и являются отражением модели реализации.

Диалоговые окна подтверждений возникают в коде программы тогда, когда программист заходит в тупик. Обычно это случается, если программе требуется принять решительные меры, а программист не может взять на себя ответственность за них. Как правило, диалоговое окно подтверждения появляется, когда исполняемая команда либо необратима, либо может привести к нежелательным последствиям.

Подтверждения перекладывают ответственность на плечи пользователя. Подтверждать действия следует тогда, когда эти действия могут привести к необратимым действиям системы (например, удаление записей из файла) или задержке во времени для подготовки данных к

следующим этапам диалога. В идеальном случае система должна предоставлять возможность восстановления утерянных данных или отказа от результатов работы.

5.6. ГИБКОСТЬ, ТАКТИЧНОСТЬ

Гибкость диалога — это мера того, насколько хорошо он соответствует различным уровням подготовки производительности труда пользователя. Такая гибкость предполагает, что диалог может подстраивать свою структуру или входные данные под конкретного пользователя.

Исследования, проведенные в Стэндфордском университете США социологами Клиффордом Нассом и Байроном Ривзом, показали, что человек реагирует на компьютер так же, как и на других людей. Для человека компьютеры ведут себя больше как люди, а не как предметы. Из этого исследования можно сделать фундаментальный вывод: если мы хотим, чтобы пользователям понравился наш программный продукт, он должен вести себя так, как ведет себя человек, приятный в общении. Программа должна вести себя как товарищ по работе, готовый поддержать коллегу.

Если программа скупа на информацию, скрывает результаты своей работы или выдает их не в том виде, как нужно конкретному пользователю, заставляет прилагать усилия для поиска обычных функций и с легкостью обвиняет пользователя в своих промахах, то вряд ли она будет ему соответствовать.

Программный продукт должен вести себя как тактичный человек (тактичность означает первоочередное внимание к интересам других).

Интерактивные продукты часто раздражают нас потому, что они бестактны, а не потому, что им не хватает каких-то возможностей. Создание тактичного продукта ничуть не сложнее, чем создание грубого или бестактного. Нужно всего-навсего реализовать себе взаимодействие, которое имитирует качества чувствительного и заботливого человека. Если программа уважает пользователя, помогает ему и подстраивается под него — она будет гибкой.

Гибкая программа:

1. Всегда интересуется пользователем и его предпочтениями, запоминает его привычки — с чем пользователь чаще работает, кому

отсылает письма, настройки, вид представления. Примером являются браузеры Firefox и Google Chrome, которые запоминают вводимую пользователем на сайтах информацию, например адреса доставки или регистрационное имя.

Программа должна изо всех сил стараться запомнить привычки пользователя и особенно команды, которые поступали от него.

2. Тактичная программа относится к пользователю как к начальнику. Она его не критикует, а если что-то не так — объяснит, что будет, и подчинится воле пользователя.

3. На конкретный вопрос не только даст ответ, но еще и предоставит сопутствующую информацию, даже если она непосредственно не связана с нашими целями.

4. Обладает здравым смыслом: не присылает счета с нулевыми или явно запредельными суммами, огромные счета за междугородние переговоры одиноким пенсионерам-долгожителям и при этом уведомляет бухгалтерию о таком событии.

5. Не выдает ненужные уведомления и техническую информацию, которая пользователя совершенно не интересует. Если возникают проблемы — программа решает их самостоятельно, не привлекая пользователя, который все равно ничего не понимает.

6. Предусмотрительна. Пользователь сообщает минимум информации, а все остальное программа доспросит или найдет сама. Если документ запущен на печать, она сообщит, что в очереди уже находится 20 документов суммарной емкостью 300 листов и обратит внимание на наличие достаточного количества бумаги в принтере. Предоставляет всю возможную информацию без значительных усилий со стороны пользователя и не задает лишних вопросов. Понятливая программа следит за действиями пользователя и на их основании предлагает ему соответствующую информацию.

Программе следует внимательно относиться и к нашим предпочтениям, запоминать их самостоятельно, без подсказки. Если пользователь всегда разворачивает окно в полный экран, приложение после нескольких сеансов должно запомнить это и всегда запускаться с уже развернутым окном. Можно подумать, что вся эта суета с памятью необязательна, — ведь проще каждый раз задавать пользователю вопросы или предложить ему все делать самому. Программисты, не задумываясь,

создают диалоговые окна (одна функция — одно окно), чтобы затребовать любую информацию, недоступную напрямую. Однако *людям не нравится, когда их допрашивают.*

Следует запоминать все, что выбирает пользователь.

Вместо жестко запрограммированного значения программа должна выбирать в качестве значения по умолчанию предыдущие настройки — и у нее будет гораздо больше шансов предоставить пользователю то, что ему требуется.

Можно перестать задавать вопросы пользователю и автоматически выбрать тот вариант, который пользователь предпочел в предыдущий раз, позволяя человеку скорректировать выбор в том случае, если программа ошиблась.

7. Тактичные программы уверены в себе и не делают из пользователя полного идиота, после подачи команды на печать файла не выводят окно с вопросом: «Вы уверены, что хотите отправить документ на печать?»

8. В случае сбоев программа не перекладывает ответственность на пользователя, который к тому же не понимает о чем идет речь и каковы будут последствия любого варианта: «Системный файл Table.dbf изменен. Сохранить? Да/Нет».

9. Работает в условиях неопределенности. Системам, работающим вручную, присуща гибкость в плане возможности выполнения работы в состоянии неопределенности или в ситуациях с неполной информацией: «Ты начни пока оформлять, а остальное я привезу попозже». Человек может обойти инструкции, автоматизированная система — нет, даже ради некоторого выигрыша. Большинство программ признают только два состояния: объект либо отсутствует, либо полностью соответствует требованиям. Промежуточные состояния не распознаются и не принимаются. В ручном варианте можно выполнять действия в нестандартной последовательности или с нарушением формальностей — с человеком можно договориться. В автоматизированной системе сговорчивость теряется, если интерфейс следует модели реализации. Программист не видит необходимости в создании промежуточных незавершенных состояний, потому что компьютер в них не нуждается. Сговорчивая программа тоже должна работать в состоянии неопределенности, частично выполнять свои функции и

следить за получением формальных данных к концу этапа. В реальном мире ограничения нарушаются постоянно. Тактичный продукт должен осознавать и учитывать этот факт.

Концепция адаптации диалога является одной из основных областей исследования взаимодействия человека и ЭВМ. Основная проблема не в том, как организовать изменения в диалоге, а в том, какие признаки надо использовать для определения необходимости внесения изменения и их сути.

В заключение можно привести слова крупнейшего на сегодняшний день американского специалиста по интерфейсам Алана Купера: «***Без сомнения, все эти решения потребуют от программистов большей работы. Это нисколько меня не волнует. Я не хочу, чтобы программисты больше работали, но, выбирая из сложности работы программистов и сложности работы пользователей, я немедленно заставляю программистов работать. Работа программиста — удовлетворить пользователя, а не наоборот***».

6. КОМПОЗИЦИЯ ИНФОРМАЦИИ НА ЭКРАНЕ

Восприятие информации с дисплея зависит от формы ее представления. Внешний вид экрана зависит от того, какие поля сообщений отображаются, в каком месте и с какими атрибутами.

Данные должны располагаться так, чтобы пользователь мог:

- просматривать экран в логической последовательности и легко находить нужную информацию
- идентифицировать связанные группы информации
- воспринимать предупреждения и сообщения об ошибках
- определять, какое действие требуется (и требуется ли вообще) для выполнения задачи.

Плотность расположения данных — понятие субъективное. Оно зависит от конкретного пользователя и задачи. Например, данные на экране, предназначенные для работы опытного оператора, располагаются плотнее, чем предназначенные для неопытного пользователя.

Если данные выводятся на несколько экранов, то нужно четко обозначить границу раздела, причем надо стараться, чтобы логическая граница совпадала с физической. Ключевая информация должна выводиться на всех экранах последовательности.

Все данные, необходимые для принятия конкретного решения, необходимо располагать на одном экране. Логически связанные данные должны представляться отдельной группой. Этого можно достичь оставляя по несколько пробелов с каждой стороны группы, либо проводя вертикальные или горизонтальные линии, либо используя различные атрибуты для полей групп.

Меню, содержащее относительно небольшой объем информации, должно смещаться в левую верхнюю часть экрана. Чтобы подчеркнуть структуру, содержимое и наименование полей, относящихся к одной группе, лучше выравнивать их по вертикали. По возможности надо выравнивать все логически связанные группы данных.

При компоновке экрана нужно определить точку начала отсчета. Фрагменты текста должны располагаться на экране так, чтобы умень-

шить время селективного поиска и количество сканирований экрана, тем самым снизив нагрузку на глаза. Информацию лучше располагать в направлении естественного движения глаза или же группами, как было сказано выше. Естественнее начать из левого верхнего угла и перемещаться слева направо и сверху вниз. Один и тот же вид информации должен появляться всегда в одном и том же месте экрана. Заголовки и содержимое полей следует не прижимать к краю экрана или поля, а располагать с некоторым отступом от границы (около 5 мм), чтобы у пользователя не возникало ощущение «сползания» к краю поверхности. Изображение на экране надо стремиться центрировать, чтобы не возникало иллюзии смещения центра вправо/влево (соответственно с другой стороны будет пустота, появится ощущение неустойчивости картины) или всплывания/опускания изображения по вертикали. Объект, расположенный в центре, предполагает большую важность, чем предметы, находящиеся по бокам. Все элементы экрана должны быть соразмерными и уравновешенными.

При использовании текстовой информации гарнитура шрифта «все заглавные» ассоциируется с задачей привлечения внимания, с повышенной громкостью голоса, криком.

Опознание изолированных букв и слов («ОШИБКА!») лучше, когда буквы заглавные, однако скорость чтения эффективнее, когда текст состоит из строчных букв. Это объясняется большей плотностью текста из строчных букв, что позволяет воспринимать большее число букв за время фиксации взглядом и тем самым повысить скорость сканирования текста. Строка или слово, набранные одними строчными буквами, производят впечатление вырванных из контекста, подвешенных в воздухе. Это может быть уместным для ссылок на другие объекты.

Гарнитура шрифта

На различимость буквенно-цифровых знаков влияет форма их начертания. В русском языке это называется **гарнитурой шрифта**, в английском — font family (или font). Основная масса шрифтов делится на группы: шрифты с засечками и без засечек (рубленые). Отдельно стоят моноширинные (fixed width) шрифты, имеющие одинаковую ширину начертания всех букв и знаков (как у печатной машинки). Все остальные шрифты попадают в категорию декоративных. Это различные стили-

зации (готика, древнерусский), рукописные шрифты (небрежно-рукописный, каллиграфически-рукописный и т.д.), шрифты с различными эффектами и спецсимволами. Внутри одной гарнитуры используют прямое и курсивное начертание, также варьируется начертание шрифта по насыщенности (bold) — полужирный, жирный, сверхжирный — и, как следствие, меняется ширина буквы. Еще один параметр шрифта — кегль, т.е. его размер (font size). Кегль шрифта измеряется по высоте заглавных букв, в пунктах (0.35 мм).

При выборе шрифтов следует руководствоваться теми же принципами, что и при выборе цвета: композиция должна содержать минимальное количество резко различных, контрастирующих шрифтов. Из всех типов шрифтов лучше всего для совместной работы подходят шрифт рубленый и шрифт с засечками. Шрифты с засечками удобнее для длительного чтения и больших объемов текста, так как засечки «удерживают» взгляд на строке; эти шрифты выглядят наряднее. Рубленые шрифты лучше всего подходят для беглого чтения и используются в заголовках, логотипах, коротких надписях. Хорошо выглядят в жирном и сверхжирном начертаниях. Шрифты рубленые и с засечками гармонично дополняют друг друга.

В тех случаях, когда необходимо выбрать два рубленых шрифта или два шрифта с засечками, нельзя выбирать два слишком похожих шрифта (как и два близких цвета).

Если есть необходимость использовать декоративный шрифт, то пусть это будет единственный шрифт на странице. Данное требование не всегда выполнимо, поэтому, если и добавлять какой-либо шрифт, то лучше рубленый, который позволит создать контраст шрифтов. В большинстве случаев декоративных шрифтов следует избегать, к тому же они быстро устаревают.

Скорость и точность восприятия текста зависят от характеристик букв, расстояний между ними и словами, от длины строк и интервалов между строками. Для длинных текстов лучше использовать одновременно прописные (заглавные) и строчные буквы, для коротких — прописные. Шрифт должен быть простым, экономичным и понятным. Чем меньше кегль, тем дальше друг от друга должны стоять буквы, и наоборот — чем крупнее шрифт, тем плотнее должны стоять буквы. Регулировка плотности набора называется трекингом (tracking).

Трекинг сдвигает или раздвигает все буквы на одну и ту же величину, исправляя кажущееся изменение плотности при изменении кегля. Тем не менее, отдельные пары букв все равно оказываются стоящими либо слишком тесно, либо слишком свободно (например, **Где**). В крупном кегле это становится очень заметно и расстояния между парами букв приходится исправлять вручную. Такая регулировка называется керингом (kerning).

Для каждого шрифта существует определенная оптимальная длина строки. Слишком длинную строку трудно держать в поле зрения, и порой нелегко найти начало следующей строки; при чтении коротких строк иногда трудно уловить смысл текста. Удобнее для чтения такая длина строки, когда фиксация ее центра позволяет периферическим зрением охватить всю строку, Как показывает опыт, оптимальная длина строки — 40 символов.

Пробелы между буквами следует устанавливать (если это возможно) с учетом формы соседних знаков, размера, плотности и насыщенности шрифта. Стандартизация пробелов между буквами и словами дает возможность сохранять определенный ритм чтения; с увеличением межстрочных интервалов облегчается чтение формул.

ЗАКЛЮЧЕНИЕ

Стремясь сделать интерфейсы как можно более простыми и удобными, необходимо учитывать человеческие интересы, возможности и ограничения восприятия информации человеком. Попытки сделать прикладную систему, акцентируя внимание на функционале и не учитывая интересы пользователя лишь на том основании, что в техническом задании имеется общая фраза «Система должна иметь удобный пользовательский интерфейс», заканчиваются либо отказом пользователя от работы с такой системой, либо вынужденным использованием этой системы с сильным желанием поскорее закрыть прикладное окно.

Непонимание руководством проекта важности удобства пользователя приводит к тому, что каждый раз при работе с системой пользователь испытывает и стресс, и чувство вины за то, что он такой «несообразительный» и пр. К сожалению, подобные эргономические решения приходится видеть и в государственных сервисах, и в попытках перевести школу на электронные ресурсы. Такие решения вызывают справедливое возмущение рядовых граждан, которые являются в рамках задачи наивными пользователями. Многие родители признаются, что работа с электронным дневником школьника вызывает у них огромное напряжение и дискомфорт и желание вернуться к прежнему бумажному дневнику, где все просто и понятно. Причина здесь одна — квалификация разработчиков не как программистов, а именно как проектировщиков пользовательского интерфейса оставляет желать лучшего.

Широкое распространение персональных компьютеров, электронных ресурсов и встроенных средств вычислительной техники требует, чтобы информационные системы становились все более дружественными, понятными для всех пользователей, в том числе и для неспециалистов в области информатики и вычислительной техники. Эта тенденция будет сохраняться и усиливаться в ближайшие годы, поэтому игнорировать такую часть системы, как интерфейс пользователя, никак нельзя.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. ISO 9241–10–1996 Ergonomic requirements for office work with visual display terminals (VDTs). P. 10. Dialogue principies.
2. ISO 9241–12–1998 Ergonomic requirements for office work with visual display terminals (VDTs). P. 12. Presentation of information.
3. ISO 9241–16–1998 Ergonomic requirements for office work with visual display terminals (VDTs). P. 16. Direct manipulation dialogues.
4. *Андреев В.Н.* Примерное содержание технического задания по разработке пользовательского интерфейса и тезисов по ведению переговоров // Интернет-источник http://www.usability.ru/toader/articles.htm Центр практичных программ.
5. *Арлов Л.* Как создать хороший интерфейс пользователя? // Интернет-источник http://www.usability.ru/toader/articles.htm Центр практичных программ.
6. *Батенькина О.В.* Дизайн пользовательского интерфейса информационных систем: Учеб. пособие. — Омск: Изд-во ОмГТУ, 2014.
7. *Вятчин К.* Определение пользовательских профилей. Электронный ресурс. [2018]. Режим доступа: http://www.sdteam.com/t15690
8. *Гарретт Дж.* Веб-дизайн: книга Джесса Гарретта. Элементы опыта взаимодействия; Пер. с англ. — СПб.: Символ-Плюс, 2008.
9. *Герасимов Ю.* Улетный интерфейс // Интернет-источник hptts://www.old.computerra.ru/194515/ Компьютерра: Улетный интерфейс (computerra.ru)
10. *Головач В.* Книги и статьи о пользовательских интерфейсах Электронный ресурс. [2018]. Режим доступа: http://www.usethics.ru/lib
11. *Головач В.* Юзабилити-тестирование по дешевке. Электронный ресурс. [2018]. Режим доступа: https://medium.com/usethics-doc/юзабилити-тестирование-по-дешевке-2e853250960f
12. *Головач В.* Дизайн пользовательского интерфейса // Интернет-источник http://www.uibook1.ru, 146 с. Центр практичных программ.
13. ГОСТ Р 56274–2014. Общие показатели и требования в эргономике. Национальный стандарт Российской Федерации: изд. офиц.: введен 2016-01-01 / НИИ экономики связи и информатики «Интерэкомс». Москва: Стандартинформ, 2015. IV. 26 с.

14. ГОСТ Р ИСО МЭК 9126–93 Информационная технология. Оценка программной продукции. Характеристики качества и руководства по их применению.
15. ГОСТ Р ИСО/МЭК 12119–2000 Информационная технология. Пакеты программ. Требования к уровню качества и тестирование.
16. *Грачев Н.Н.* Психология инженерного труда. — М.: Высшая школа, 1998.
17. *Гультяев А.*, *Машин И.* Проектирование и дизайн пользовательского интерфейса. — СПб.: Корона принт, 2000.
18. *Дениинг В.*, *Эссиг Г.*, *Маас С.* Диалоговые системы «Человек-ЭВМ». Адаптация к требованиям пользователя. — М.: Мир, 1984.
19. *Раскин Д.* Интерфейс: новые направления в проектировании компьютерных систем; Пер. с англ. — СПб.: Символ-Плюс, 2003.
20. *Донской М.* Пользовательский интерфейс // Интернет-источник http://www.usability.ru/toader/articles/user_interface.htm Центр практичных программ.
21. *Жарков С.В.* Shareware: профессиональная разработка и продвижение программ. — СПб.: БХВ-Петербург, 2003.
22. Интеллект человека и программы ЭВМ / Под ред. О.К. Тихомирова. — М., 1979. С. 230.
23. *Константин Л.*, *Локвуд Л.* Разработка программного обеспечения. — СПб.: Питер, 2004.
24. *Коутс Р.*, *Влейминк И.* Интерфейс «Человек-Компьютер». — М.: Мир, 1990.
25. *Кузнецов А.* Эмоциональный дизайн или тайна четвертой волны. Электронный ресурс. [2017]. Режим доступа: https://uexpert.ru/emotsionalnyj-dizajn-ilitajna-chetvyortoj-volny/
26. *Купер А.*, *Рейман Р.*, *Кронин Д.* Алан Купер об интерфейсе. Основы проектирования взаимодействия; Пер. с англ. — СПб.: Символ-Плюс, 2010.
27. *Латышев В.Л.*, *Клыпина И.А.* Представление информации в системе «Человек-компьютер». — М.: Изд-во МАИ, 1993.
28. *Леонтьев А.Н.* Деятельность. Сознание. Личность. — 2-е изд. — М.: Смысл: Академия, 2005.
29. *Лукин В.В.*, *Лукин В.Н.*, *Лукин Т.В.* Технология разработки программного обеспечения. — 4-е изд. — М.: Вузовская книга, 2019. С. 294.
30. *Лукин В.Н.*, *Дзюбенко А.Л.*, *Чечиков Ю.Б.* Подходы к разработке пользовательского интерфейса // Программирование. 2020. № 5. С. 16–24.

31. *Лукин В.Н.*, *Зотова А.А.* Пользовательский интерфейс: история и современность. «Новые информационные технологии» // Тез. докл. XV Международной студенческой школы-семинара. — М.: МИЭМ, 2007. С. 50–55.
32. *Лукин В.Н.*, *Скобеева М.В.*, *Чечиков Ю.Б. и др.* Автоматизированная биохимическая лаборатория. Перспективное развитие // Лабораторное дело. 1989. № 9. С. 26–28.
33. *Минанси М.* Графический интерфейс пользователя. Секреты проектирования. — М.: Мир, 1996.
34. *Нильсен Я.* Веб-дизайн. Книга Якоба Нильсена; Пер. с англ. — СПб.: Символ-Плюс, 2003.
35. *Нильсен Я.* Элементарные основы юзабилити. Электронный ресурс. [2018]. Режим доступа https://www.promo-webcom.by/analytics/usability/1429-elementarnyie-osnovyi-yuzabiliti/
36. *Норман Д.* Дизайн вещей будущего; Пер. с англ. — М.: ИД «Вильямс», 2006.
37. *Норман Д.* Дизайн привычных вещей; Пер. с англ. — М.: ИД «Вильямс», 2006.
38. *Платт Д.* Софт-отстой и что с этим делать. — СПб. — М.: Симбо, 2008.
39. *Раскин Дж.* Интерфейс: новые направления в проектировании компьютерных систем. — СПб.: Символ-Плюс, 2005.
40. *Седельников А.* Пять распространенных ошибок при разработке интерфейсов программ // Интернет-источник http://www.usability.ru/toader/mycolumn/five_errors.htm Центр практичных программ.
41. *Соловьев С.В.*, *Цой Р.И.*, *Гринкруг Л.С.* Технология разработки прикладного программного обеспечения. https://monographies.ru/en/book/view?id=141
42. *Сполский Дж.* Программистам о разработке пользовательских интерфейсов // Интернет-источник http://www.usability.ru/toader/articles/uid4p_1.htm Центр практичных программ.
43. *Сугак Е.* Эргономические аспекты проектирования пользовательского интерфейса: Дисс. ... канд. психолог. наук (специальность 19.00.03 — психология труда, инженерная психология). — М.: МГУ, 2005.
44. *Тонгаззини Б.* Вежливый интерфейс или принципы создания диалогов // Интернет-источник http://www.usability.ru/toader/articles.htm Центр практичных программ.
45. *Уолтер А.* Эмоциональный веб-дизайн; Пер. с англ. — М.: Манн, Иванов и Фербер, 2012.

46. Хрестоматия по инженерной психологии / Сост.: Б.А. Душков, Б.Ф. Ломов, Б.А. Смирнов / Под ред. Б.А. Душкова. — М.: Высшая школа, 1991.
47. Человеко-машинный интерфейс. Правила организации http://genew.ru/cheloveko-mashinnij-interfejs-pravila-organizacii.html?page=6
48. *Шнейдерман Б.* Психология программирования: Человеческие факторы в вычислительных и информационных системах. — М.: Радио и связь, 1984.
49. *Шнейдерман Б.* Разработка пользовательского интерфейса: стратегии эффективного взаимодействия человека и компьютера. — М.: Радио и связь, 2016.
50. Эмоциональный дизайн с примерами, перевод материала Smashing magazine, https://naikom.ru/blog/archives/6382
51. Статьи по юзабилити www.usability.ru/toader/articles.htm

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3

1. ВЗАИМОДЕЙСТВИЕ ЧЕЛОВЕКА И ЭВМ 6
   1.1. Основные психологические факторы взаимодействия человека и ЭВМ 6
   1.2. Понятие интерфейса пользователя 12
   1.3. Распределение функций между человеком и ЭВМ 14

2. ПОЛЬЗОВАТЕЛИ, УДОБСТВО, РАЗРАБОТКА 16
   2.1. Классификация пользователей 16
   2.2. «Наивный» пользователь 17
   2.3. Удобство для пользователя 17
   2.4. Ожидания пользователя 19
   2.5. Психологические основы разработки интерфейса 21
   2.6. Привычки пользователя 23
   2.7. Подходы к проектированию интерфейсов 24
   2.8. Основные методологии разработки интерфейсов 27
      2.8.1. Дизайн, ориентированный на деятельность (ACD) 27
      2.8.2. Целеориентированный дизайн (Goal-oriented design) 28
      2.8.3. Дизайн, ориентированный на пользователя (UCD) 28
   2.9. Основные рекомендации по проектированию пользовательского интерфейса 29
      2.9.1. Восемь «золотых» правил Шнейдермана 29
      2.9.2. Десять эвристических правил Якоба Нильсена 30
      2.9.3. Принципы Usage Centered Design 32
   2.10. Процесс разработки интерфейса 33

3. МОДЕЛИ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА 36
   3.1. Парадигмы интерфейса 36
   3.2. Модель реализации 37
   3.3. Модель пользователя 38
   3.4. Модель представления 39

4. МЕТАФОРЫ И ИДИОМЫ 41

5. КРИТЕРИИ ХОРОШЕГО ДИАЛОГА 45
5.1. Естественность 45
5.2. Последовательность/стандартизация 47
5.3. Краткость 50
5.4. Поддержка пользователей 51
5.5. Сообщения об ошибках 52
5.6. Гибкость, тактичность 60

6. КОМПОЗИЦИЯ ИНФОРМАЦИИ НА ЭКРАНЕ 64

ЗАКЛЮЧЕНИЕ 68

БИБЛИОГРАФИЧЕСКИЙ СПИСОК 69

Тем. план 2022, ч. 2, поз. 8

Чечиков Юрий Борисович
Лукин Владимир Николаевич
Секретарёв Виталий Евгеньевич

ОСНОВЫ ЭРГОНОМИКИ ИНТЕРАКТИВНЫХ СИСТЕМ

Редактор *М.С. Винниченко*
Компьютерная верстка *О.А. Пелипенко*

Сдано в набор 01.12.2022. Подписано в печать 12.01.2023.
Бумага писчая. Формат 60×84 1/16. Печать цифровая.
Усл. печ. л. 4,42. Уч.-изд. л. 4,75. Тираж 100 экз.
Заказ 279/1081.

Издательство МАИ
(МАИ), Волоколамское ш., д. 4
Москва, А-80, ГСП-3 125993

Типография Издательства МАИ
(МАИ), Волоколамское ш., д. 4
Москва, А-80, ГСП-3 125993